The Numerov Method for the One-Dimensional Schrödinger Equation

Course notes — free particle, square potential barrier and harmonic oscillator, with normalisation, error analysis, convergence studies, and Python 3 / Fortran 90 codes

November 2026

Table of Contents

How to use these notes 4

Part I — The Numerov method 5

The mathematical problem 5

Derivation 5

Physical and geometrical interpretation 6

Error of the method 6

Strengths and faults (limitations) of the Numerov method 7

How eigenvalues are found: the shooting idea 7

Generic logic chart of the Numerov integrator 8

Tools for verification and error analysis used throughout 8

Part II — Problem 1: The free particle 10

Physics and analytical solution 10

Numerical scheme 10

Logic chart 11

Python 3 code 11

Fortran 90 code 14

Results, normalisation and verification 16

Error analysis and convergence 18

Part III — Problem 2: The square potential barrier 19

Physics and analytical solution 19

Numerical scheme 19

Logic chart 20

Python 3 code 21

Fortran 90 code 23

Results, normalisation and verification 25

Error analysis and convergence 26

Part IV — Problem 3: The quantum harmonic oscillator 28

Physics and analytical solution 28

Numerical scheme 28

Logic chart 29

Python 3 code 29

Fortran 90 code 32

Results, normalisation and verification 36

Error analysis and convergence 37

Part V — Cross-check of the Fortran and Python implementations 38

Part VI — Summary 39
Part VII — Exercises 40
References 41

How to use these notes

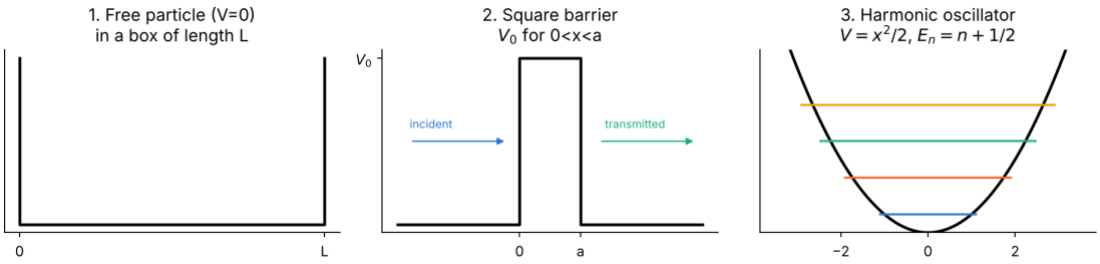
These notes form a self-contained study unit on one of the most useful algorithms of computational quantum mechanics. Each of the three physical problems is treated in exactly the same order, so that the student can see how the same tool is adapted to different physics:

1. **Physics and analytical solution** — the exact result we will test against.
2. **Numerical scheme** — the discretised equations, boundary conditions and the algorithmic strategy (shooting, matching, scattering).
3. **Logic chart** — a flowchart of the program.
4. **Codes** — complete Python 3 and Fortran 90 programs. Both versions implement the same algorithm and give the same numbers to all printed digits.
5. **Results, normalisation and verification.**
6. **Error analysis and convergence plots.**

Units used throughout: $\hbar = m = 1$ (and $\omega = 1$ for the oscillator). The time-independent Schrödinger equation then reads

$$-\frac{1}{2}\,\psi''(x) + V(x)\,\psi(x) = E\,\psi(x) \quad \Leftrightarrow \quad \psi''(x) = -k^2(x)\,\psi(x), \qquad k^2(x) = 2\,[E - V(x)].$$

In regions where $E > V$ (classically allowed) $k^2 > 0$ and ψ oscillates; where $E < V$ (classically forbidden) $k^2 < 0$ and ψ grows or decays exponentially. This sign change is the source of most of the practical difficulties discussed below.



The three model potentials studied in this unit.

Software. Python 3 with NumPy and Matplotlib; any Fortran 90 compiler (e.g. `gfortran`). Compile and run each Fortran program with

```
gfortran -O2 -o p1 p1_free_particle.f90 && ./p1
gfortran -O2 -o p2 p2_square_barrier.f90 && ./p2
gfortran -O2 -o p3 p3_harmonic_oscillator.f90 && ./p3
python3 plot_fortran_results.py      # plots the .dat files written by the Fortran codes
```

and each Python program with `python3 p1_free_particle.py` etc.

Part I — The Numerov method

The mathematical problem

Numerov's method (Boris Numerov, 1924) integrates second-order linear ODEs **without a first-derivative term**,

$$y''(x) = -g(x)\,y(x) + s(x),$$

which is exactly the structure of the one-dimensional (or radial) Schrödinger equation with $g = k^2$, $s = 0$. It is a three-point, two-step, **fourth-order** method that costs essentially the same as the simplest second-order finite-difference scheme.

Derivation

Introduce a uniform grid $x_n = x_0 + nh$ and write $y_n = y(x_n)$. Taylor-expand forwards and backwards:

$$y_{n\pm1} = y_n \pm h{y'}_n + \frac{h^2}{2}{y''}_n \pm \frac{h^3}{6}{y'''}_n + \frac{h^4}{24}y_n^{(4)} \pm \frac{h^5}{120}y_n^{(5)} + \frac{h^6}{720}y_n^{(6)} + \cdots$$

Adding the two expansions, **all odd derivatives cancel**:

$$y_{n+1} - 2y_n + y_{n-1} = h^2{y''}_n + \frac{h^4}{12}y_n^{(4)} + \frac{h^6}{360}y_n^{(6)} + O(h^8).$$

Truncating after $h^2{y''}_n$ gives the ordinary second-order central difference. Numerov's trick is to keep the h^4 term and *eliminate* the fourth derivative by applying the same central difference to y'' itself:

$$y_n^{(4)} = \frac{{y''}_{n+1} - 2{y''}_n + {y''}_{n-1}}{h^2} - \frac{h^2}{12}y_n^{(6)} + O(h^4).$$

Substituting this into the previous equation:

$$y_{n+1} - 2y_n + y_{n-1} = \frac{h^2}{12}({y''}_{n+1} + 10\,{y''}_n + {y''}_{n-1}) - \frac{h^6}{240}\,y_n^{(6)} + O(h^8).$$

Now use the differential equation, $y'' = -g\,y$, at the three nodes. Collecting the unknown y_{n+1} on the left gives the **Numerov recursion**:

$$\boxed{y_{n+1}\left(1 + \frac{h^2}{12}g_{n+1}\right) = 2\,y_n\left(1 - \frac{5h^2}{12}g_n\right) - y_{n-1}\left(1 + \frac{h^2}{12}g_{n-1}\right)}$$

With the abbreviation $c_n = 1 + h^2 g_n/12$ this is written compactly — and this is literally the line that appears in every code of this unit —

$$y_{n+1} = \frac{(12 - 10\,c_n)\,y_n - c_{n-1}\,y_{n-1}}{c_{n+1}}.$$

The same formula run "backwards" (solve for y_{n-1}) integrates from right to left; this is used in Problems 2 and 3.

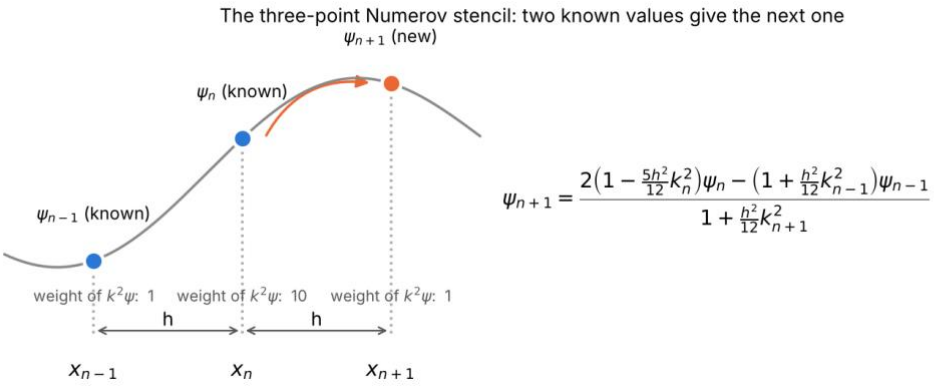


Illustration of the Numerov stencil. Two known values (blue) and the potential at three points give the new value (orange). The weights 1–10–1 of the $k^2\psi$ terms are what raise the accuracy from second to fourth order.

Physical and geometrical interpretation

- The left-hand side $y_{n+1} - 2y_n + y_{n-1}$ measures the **curvature** of ψ on the grid. The Schrödinger equation says that curvature equals $-k^2\psi$: the wavefunction bends *towards* the axis where $E > V$ (oscillation) and *away* from it where $E < V$ (exponential behaviour).
- Plain finite differences evaluate that curvature only at the centre point. Numerov averages it with weights $1:10:1$ — a Simpson-like weighting — so that the leading error term cancels.
- Because the scheme is linear in y, the overall scale of the starting values is irrelevant: a wavefunction is defined up to a constant, which is fixed afterwards by **normalisation**.

Error of the method

Local truncation error. One step commits an error

$$\tau = -\frac{h^6}{240}\,\psi^{(6)}(x_n) + O(h^8).$$

Global error. The local error is a defect in a *second-difference* equation. Dividing by h^2 it acts like a spurious source of size $\tau/h^2 = O(h^4)$ in ψ'', which, integrated twice over a fixed interval, produces a global error

$$\varepsilon_{\text{global}} = O(h^4).$$

Halving h therefore reduces the error by $2^4 = 16$. The observed ratio 16 is the fingerprint we look for in all convergence tables of this unit.

Starting-value error. An error δ in the second starting value y_1 is equivalent to an error δ/h in the initial slope, which propagates to a global error $O(\delta/h)$. To keep fourth-order accuracy one needs $\delta = O(h^5)$. A first-order Taylor start $y_1 = y_0 + hy'_0$ has $\delta = O(h^3)$ and spoils the method down to $O(h^2)$ — demonstrated numerically in Problem 1.

Phase (dispersion) error. For constant k the exact discrete solution is $e^{\pm ik_h x}$ with

$$\cos(k_h h) = \frac{1-\frac{5}{12}(kh)^2}{1+\frac{1}{12}(kh)^2} \quad \Rightarrow \quad k_h = k\left[1+\frac{(kh)^4}{480}+\cdots\right].$$

The numerical wave runs slightly faster than the true wave; after a distance X the phase error is $k^5h^4X/480$. Rule of thumb: use at least 20–30 grid points per de Broglie wavelength.

Round-off error. Every step adds an error of order machine precision $\epsilon \approx 10^{-16}$ times $|\psi|$. Accumulated over $N \propto 1/h$ steps of a second-order recursion, the round-off contribution grows roughly like ϵ/h^2. Total error $\approx Ah^4 + B\epsilon/h^2$ therefore has a minimum; below an optimal h (here around $h \sim 10^{-3}$) refining the grid makes things *worse*. All convergence plots below show this floor at the level 10^{-11}–10^{-12}.

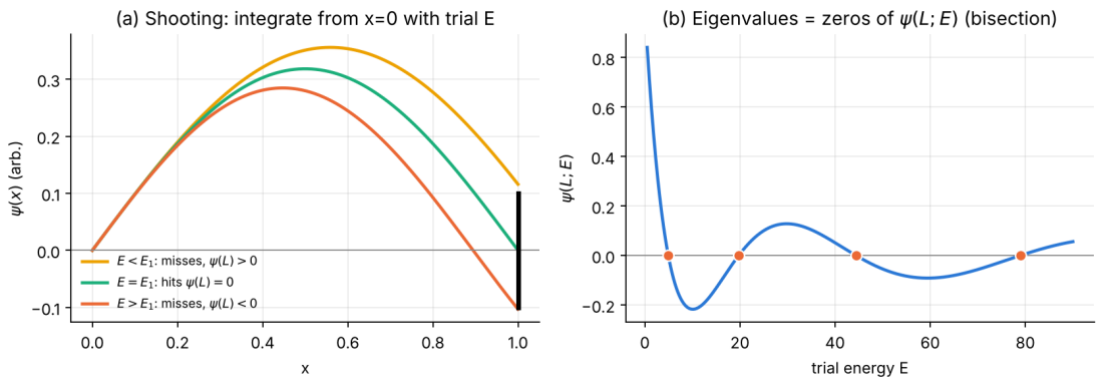
Strengths and faults (limitations) of the Numerov method

The method is elegant and accurate, *nevertheless* it has well-defined faults which every user must know. Each of them is met — and cured — in one of the three problems.

#	Fault / limitation	Consequence	Cure used in this unit
1	Only for $y'' = -gy + s$ (no y' term)	Equations with first derivatives must first be transformed (e.g. $R(r) = u(r)/r$ in the radial problem)	All three problems already have this form
2	Not self-starting: needs y_0 **and** y_1	An inaccurate y_1 degrades the order from 4 to 2	Exact or asymptotic start values (Problem 1, Part A)
3	Assumes a smooth potential: $\psi^{(6)}$ must exist	A step in V (barrier edge) drops the order to 2	Edge nodes, one-sided V, jump correction (Problem 2)
4	Unstable when integrating **into** a classically forbidden region	Growing exponential $e^{+\kappa x}$ swamps the physical decaying one	Two-sided shooting with matching (Problem 3)
5	Requires $c_n = 1 + h^2k_n^2/12 \neq 0$	For deep forbidden regions with coarse h the scheme breaks down	Choose h so that $h^2k^2 \gg -12$ (i.e. small in magnitude)
6	Uniform grid only	Singular or long-range potentials need very many points	(Logarithmic grid + change of variable — not needed here)
7	It is an *integrator*, not an eigenvalue solver	Energies must be found by shooting + root finding; levels can be missed	Energy scan + bisection + node counting (Problems 1, 3)
8	Round-off floor	Error stops decreasing for very small h	Use h near the optimum; Richardson extrapolation
9	Free-particle (continuum) states are not square-integrable	$\int \psi^*\psi\, dx$ diverges	Box normalisation (Problem 1) or flux normalisation (Problem 2)

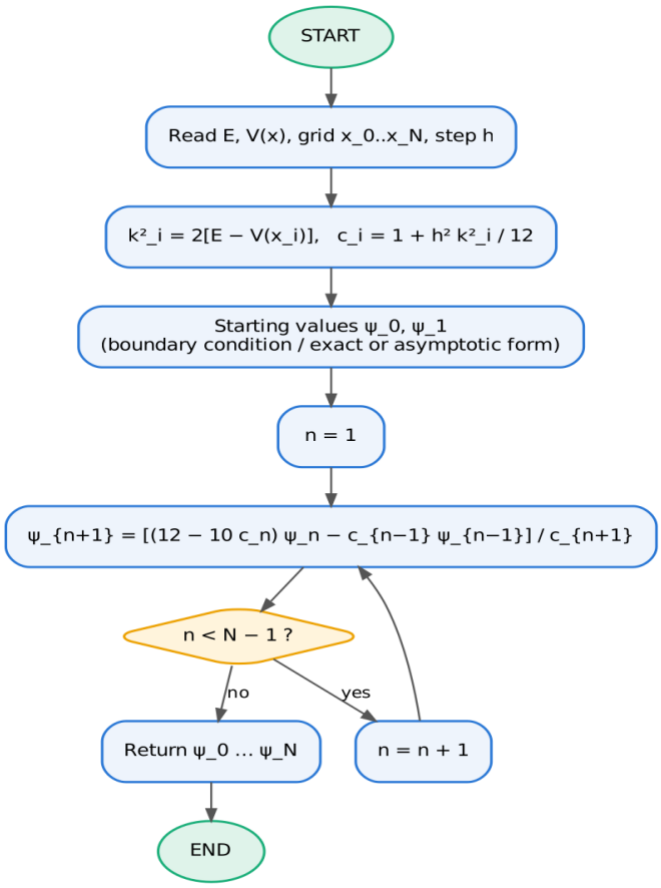
How eigenvalues are found: the shooting idea

For a bound state the energy is unknown. We guess E, integrate from one boundary where ψ is known, and check whether the other boundary condition is satisfied. The miss distance is a continuous function of E whose zeros are the eigenvalues. They are bracketed by a coarse scan and refined by bisection, which halves the uncertainty in each step and cannot fail once a sign change is found.



The shooting method for a particle in a box: (a) trial solutions for three energies, (b) the end value $\psi(L;E)$ as a function of the trial energy; its zeros are the eigenvalues.

Generic logic chart of the Numerov integrator



Logic chart of the Numerov integrator used as a subroutine by all three programs.

Tools for verification and error analysis used throughout

Normalisation with the composite Simpson rule (error $O(h^4)$, consistent with Numerov):

$$\int_{x_0}^{x_N} |\psi|^2\,dx \approx \frac{h}{3}\left[\,|\psi_0|^2 + 4\sum_{\text{odd } n}|\psi_n|^2 + 2\sum_{\text{even } n}|\psi_n|^2 + |\psi_N|^2\right], \qquad \psi \leftarrow \frac{\psi}{\sqrt{\int |\psi|^2 dx}}.$$

Observed order of convergence. If $\varepsilon(h) \approx Ch^p$ then $\log\varepsilon = \log C + p\log h$; p is obtained as the least-squares slope of a log–log plot, or from two successive grids as $p = \log_2[\varepsilon(h)/\varepsilon(h/2)]$.

Richardson extrapolation. Knowing $p = 4$, two calculations with steps h and $h/2$ are combined into an improved estimate

$$E_R = \frac{2^4 E(h/2) - E(h)}{2^4 - 1} = \frac{16\,E(h/2) - E(h)}{15},$$

which removes the leading h^4 error term.

Other checks. Orthogonality $\langle \psi_i | \psi_j \rangle = \delta_{ij}$, the virial theorem (Problem 3) and flux conservation $R + T = 1$ (Problem 2).

Part II — Problem 1: The free particle

Physics and analytical solution

For $V(x) = 0$ the Schrödinger equation is $\psi'' = -k^2\psi$ with $k = \sqrt{2E}$. Its general solution is

$$\psi(x) = Ae^{ikx} + Be^{-ikx} = C\sin kx + D\cos kx, \qquad E = \frac{k^2}{2}.$$

Two separate questions are studied.

Part A — the unbounded free particle (initial-value problem). With $\psi(0) = 0$ and a chosen k the exact solution is $\psi(x) = \sin kx$ (amplitude fixed to 1). Such a state is **not normalisable** on the whole real line: $\int_{-\infty}^{\infty} |\psi|^2\, dx = \infty$. Continuum states are normalised either to a delta function, $\langle k|k'\rangle = \delta(k - k')$, or by confining the particle to a large box. Part A therefore tests only the *accuracy* of the integration.

Part B — free particle in a box $[0, L]$ (eigenvalue problem). Imposing $\psi(0) = \psi(L) = 0$ (infinite walls) quantises k:

$$k_n = \frac{n\pi}{L}, \qquad E_n = \frac{n^2\pi^2}{2L^2}, \qquad \psi_n(x) = \sqrt{\frac{2}{L}}\sin\frac{n\pi x}{L}, \qquad n = 1,2,3,\ldots$$

These functions are normalised, $\int_0^L \psi_n^2\, dx = 1$, and orthogonal. We use $L = 1$, so $E_1 = \pi^2/2 = 4.934802200545\ldots$

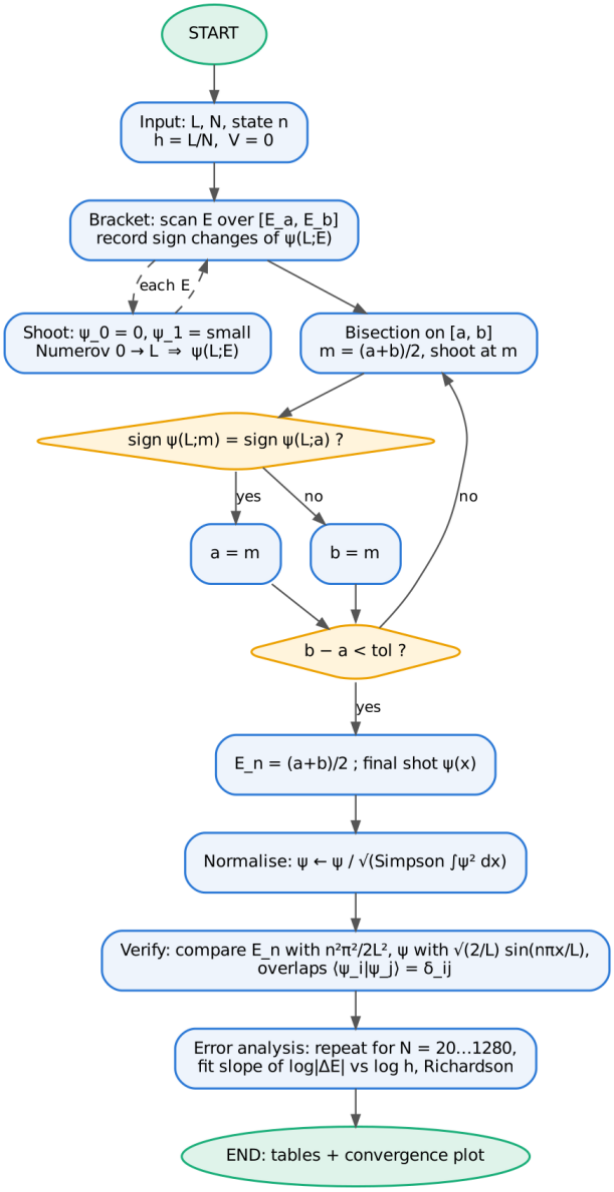
Numerical scheme

- Grid: $x_j = jh$, $h = L/N$, $j = 0, \ldots, N$; $k_j^2 = 2E$ (constant).
- **Part A.** Starting values $\psi_0 = 0$ and either the exact $\psi_1 = \sin kh$ or the Taylor approximation $\psi_1 = kh$ (error $k^3h^3/6$). Integrate forward to $X = 10$ and record $\max_j |\psi_j - \sin kx_j|$ for $N = 25 \ldots 3200$.
- **Part B.** For a trial E: $\psi_0 = 0$, $\psi_1 = \eta$ (any small number — the scale is irrelevant), integrate to $x_N = L$. The eigenvalue condition is

  $$F(E) \equiv \psi_N(E) = 0.$$

  Scan E to find a sign change of F, refine by bisection to relative tolerance 10^{-13}, integrate once more at the converged E, normalise with Simpson, fix the arbitrary sign, and compare with the exact ψ_n.

Logic chart



Logic chart of the free-particle program (Part B, eigenvalue search; Part A is the integrator of Part I called with exact or Taylor start values).

Python 3 code

File: `p1_free_particle.py`

```
#!/usr/bin/env python3
"""
Problem 1 -- FREE PARTICLE solved with the Numerov method
=========================================================
Units: hbar = m = 1  ->  psi''(x) = -k^2(x) psi(x),  k^2(x) = 2 [E - V(x)],  V = 0

Part A  (initial-value problem, unbounded free particle)
        psi(0) = 0, energy E = k^2/2 given  ->  exact solution psi(x) = sin(kx).
        We integrate with Numerov and measure the global error vs. step h.
        We also show the "starting-value fault": using a Taylor start
        psi(h) = k h (instead of the exact sin(kh)) degrades the global order.

Part B  (free particle confined to a box [0, L] -- hard walls)
        A free particle is NOT normalisable on the whole real line (plane waves),
        so to perform a normalisation check we confine it:
        psi(0) = psi(L) = 0,  E_n = n^2 pi^2 / (2 L^2),  psi_n = sqrt(2/L) sin(n pi x / L).
        Eigenvalues are found by shooting + bisection, eigenfunctions are
        normalised with Simpson's rule and compared with the analytic ones.
```

```
Outputs: printed tables + figures  p1_wavefunctions.png, p1_convergence.png
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4"]   # fixed series colours


# ---------------------------------------------------------------- Numerov core
def numerov(k2, psi0, psi1, h):
    """Integrate psi'' = -k2(x) psi on a uniform grid.
    k2   : array of k^2 at every grid point
    psi0, psi1 : two starting values
    Returns psi on the whole grid.
      psi_{n+1} (1 + h^2 k2_{n+1}/12) = 2 psi_n (1 - 5 h^2 k2_n/12)
                                      - psi_{n-1} (1 + h^2 k2_{n-1}/12)
    """
    N = len(k2)
    c = 1.0 + h * h * k2 / 12.0                 # Numerov weights
    psi = np.zeros(N, dtype=np.result_type(psi0, psi1, float))
    psi[0], psi[1] = psi0, psi1
    for n in range(1, N - 1):
        psi[n + 1] = ((12.0 - 10.0 * c[n]) * psi[n] - c[n - 1] * psi[n - 1]) / c[n + 1]
    return psi


def simpson(y, h):
    """Composite Simpson rule (needs an even number of intervals)."""
    n = len(y) - 1
    if n % 2:
        raise ValueError("Simpson needs an even number of intervals")
    return h / 3.0 * (y[0] + y[-1] + 4.0 * y[1:-1:2].sum() + 2.0 * y[2:-1:2].sum())


# ---------------------------------------------------------------- Part A
def part_A(k=2.0, X=10.0, N=200, exact_start=True):
    h = X / N
    x = np.linspace(0.0, X, N + 1)
    k2 = np.full_like(x, k * k)                          # V = 0
    psi1 = np.sin(k * h) if exact_start else k * h       # starting value
    psi = numerov(k2, 0.0, psi1, h)
    err = np.max(np.abs(psi - np.sin(k * x)))
    return x, psi, err


# ---------------------------------------------------------------- Part B
def shoot_box(E, L, N):
    h = L / N
    k2 = np.full(N + 1, 2.0 * E)
    psi = numerov(k2, 0.0, 1e-3 * h, h)                  # psi(0)=0, small slope
    return psi


def box_eigen(n, L=1.0, N=200, tol=1e-13):
    """n-th eigenvalue (n = 1,2,..): bracket by scanning psi(L), then bisection."""
    Eexact = (n * np.pi / L) ** 2 / 2.0
    # bracket: scan an interval around the estimate (generic scan works too)
    Ea, Eb = 0.5 * Eexact * (1 - 1 / n) + 1e-3, Eexact * (1 + 1.0 / n)
    grid = np.linspace(Ea, Eb, 200)
    f = np.array([shoot_box(E, L, N)[-1] for E in grid])
    i = np.where(np.sign(f[:-1]) != np.sign(f[1:]))[0]
    # pick the root closest to the estimate (the n-th one)
    j = i[np.argmin(np.abs(grid[i] - Eexact))]
    a, b, fa = grid[j], grid[j + 1], f[j]
    while b - a > tol * max(1.0, abs(a)):
        m = 0.5 * (a + b)
        fm = shoot_box(m, L, N)[-1]
        if np.sign(fm) == np.sign(fa):
            a, fa = m, fm
        else:
            b = m
    E = 0.5 * (a + b)
    psi = shoot_box(E, L, N)
    h = L / N
    psi /= np.sqrt(simpson(psi ** 2, h))                 # NORMALISATION
    x = np.linspace(0, L, N + 1)
    exact = np.sqrt(2 / L) * np.sin(n * np.pi * x / L)
    if np.dot(psi, exact) < 0:
        psi = -psi                                       # fix arbitrary sign
```

```
    return E, Eexact, x, psi, exact


def slope(h, e):
    p = np.polyfit(np.log(h), np.log(e), 1)
    return p[0]


def main():
    print("=" * 70)
    print("PART A : free particle, psi(0)=0, k = 2, exact solution sin(kx)")
    print("=" * 70)
    Ns = [25, 50, 100, 200, 400, 800, 1600, 3200]
    hA, eA, eT = [], [], []
    print(f"{'N':>6} {'h':>10} {'err(exact start)':>18} {'ratio':>7} {'err(Taylor start)':>18} {'ratio':>7}")
    for i, N in enumerate(Ns):
        _, _, e1 = part_A(N=N, exact_start=True)
        _, _, e2 = part_A(N=N, exact_start=False)
        r1 = eA[-1] / e1 if eA else np.nan
        r2 = eT[-1] / e2 if eT else np.nan
        hA.append(10.0 / N); eA.append(e1); eT.append(e2)
        print(f"{N:6d} {10.0/N:10.5f} {e1:18.3e} {r1:7.2f} {e2:18.3e} {r2:7.2f}")
    print(f"observed order (exact start)  p = {slope(hA, eA):.2f}   (theory 4)")
    print(f"observed order (Taylor start) p = {slope(hA, eT):.2f}   (theory 2: start error O(h^3) / h)")

    print("\n" + "=" * 70)
    print("PART B : free particle in a box L = 1  (E_n = n^2 pi^2 / 2)")
    print("=" * 70)
    print(f"{'n':>3} {'E_numerov':>18} {'E_exact':>18} {'rel.err':>11} {'norm':>14} {'max|psi-exact|':>15}")
    states = {}
    for n in (1, 2, 3, 4):
        E, Ex, x, psi, ex = box_eigen(n, N=200)
        states[n] = (x, psi, ex)
        print(f"{n:3d} {E:18.12f} {Ex:18.12f} {abs(E-Ex)/Ex:11.3e} "
              f"{simpson(psi**2, x[1]-x[0]):14.12f} {np.max(abs(psi-ex)):15.3e}")
    # orthogonality check
    O = np.array([[simpson(states[i][1] * states[j][1], states[1][0][1]) for j in (1, 2, 3, 4)]
                  for i in (1, 2, 3, 4)])
    print("overlap matrix <psi_i|psi_j>  (should be identity):")
    print(np.array2string(O, precision=2, suppress_small=False, formatter={'float': lambda v: f"{v: .1e}"}))

    print("\nConvergence of the eigenvalues in the box:")
    NsB = [20, 40, 80, 160, 320, 640, 1280]
    hB = [1.0 / N for N in NsB]
    errB = {}
    for n in (1, 2, 3):
        errB[n] = []
        for N in NsB:
            E, Ex, *_ = box_eigen(n, N=N)
            errB[n].append(abs(E - Ex) / Ex)
        print(f"  n={n}: rel. errors", " ".join(f"{e:.2e}" for e in errB[n]),
              f"-> order {slope(hB[:5], errB[n][:5]):.2f}")

    # Richardson extrapolation of E_1 (error ~ h^4)
    E_h = box_eigen(1, N=40)[0]
    E_h2 = box_eigen(1, N=80)[0]
    E_R = (16 * E_h2 - E_h) / 15
    print(f"\nRichardson (n=1): E(h)={E_h:.12f} E(h/2)={E_h2:.12f} "
          f"E_R={E_R:.12f}  exact={np.pi**2/2:.12f}  err_R={abs(E_R-np.pi**2/2):.2e}")

    # ---------------------------------------------------------- figures
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    x, psi, _ = part_A(N=100)
    xe = np.linspace(0, 10, 1000)
    ax[0].plot(xe, np.sin(2 * xe), color="#888888", lw=1, label="exact  sin(kx)")
    ax[0].plot(x[::3], psi[::3], "o", ms=4, color=C[0], label="Numerov (N=100)")
    ax[0].set(xlabel="x", ylabel=r"$\psi(x)$", title="(a) Free particle, k = 2")
    ax[0].legend(frameon=False, fontsize=8, loc="lower left")
    for i, n in enumerate((1, 2, 3, 4)):
        x, psi, ex = states[n]
        ax[1].plot(x, ex, color="#888888", lw=1)
        ax[1].plot(x[::8], psi[::8], "o", ms=3.5, color=C[i], label=f"n={n}")
    ax[1].set(xlabel="x", ylabel=r"$\psi_n(x)$", title="(b) Free particle in a box (normalised)")
    ax[1].legend(frameon=False, fontsize=8, ncol=4, loc="lower center")
    for a in ax:
        a.spines[["top", "right"]].set_visible(False); a.grid(alpha=.25)
    fig.tight_layout(); fig.savefig("p1_wavefunctions.png", dpi=170)

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    ax[0].loglog(hA, eA, "o-", color=C[0], lw=2, label="exact start  sin(kh)")
    ax[0].loglog(hA, eT, "s-", color=C[1], lw=2, label="Taylor start  kh")
    h = np.array(hA)
```

```
    ax[0].loglog(h, eA[0] * (h / h[0]) ** 4, "--", color="#888888", lw=1, label=r"$\propto h^4$")
    ax[0].loglog(h, eT[0] * (h / h[0]) ** 2, ":", color="#888888", lw=1, label=r"$\propto h^2$")
    ax[0].set(xlabel="step h", ylabel=r"max $|\psi-\psi_{exact}|$", title="(a) Global error, free particle")
    ax[0].legend(frameon=False, fontsize=8)
    for i, n in enumerate((1, 2, 3)):
        ax[1].loglog(hB, errB[n], "o-", color=C[i], lw=2, label=f"E_{n}")
    hb = np.array(hB)
    ax[1].loglog(hb, errB[1][0] * (hb / hb[0]) ** 4, "--", color="#888888", lw=1, label=r"$\propto h^4$")
    ax[1].set(xlabel="step h", ylabel="relative error of $E_n$", title="(b) Eigenvalue convergence, box")
    ax[1].legend(frameon=False, fontsize=8)
    for a in ax:
        a.spines[["top", "right"]].set_visible(False); a.grid(alpha=.25, which="both")
    fig.tight_layout(); fig.savefig("p1_convergence.png", dpi=170)
    print("\nfigures written: p1_wavefunctions.png, p1_convergence.png")


if __name__ == "__main__":
    main()
```

Fortran 90 code

File: `p1_free_particle.f90`

```
!======================================================================
! Problem 1 -- FREE PARTICLE solved with the Numerov method (Fortran 90)
! Units hbar = m = 1 :  psi'' = -k^2 psi ,  k^2 = 2(E - V),  V = 0
!
! Part A : initial-value problem psi(0)=0, k=2  -> exact psi = sin(kx)
!          global error vs h, exact start vs Taylor start psi(h)=kh
! Part B : free particle in a box [0,L], psi(0)=psi(L)=0
!          shooting + bisection, Simpson normalisation, comparison with
!          sqrt(2/L) sin(n pi x/L), convergence of E_n,  Richardson
! Output : screen tables + p1_partA.dat, p1_partB.dat, p1_states.dat
! Compile: gfortran -O2 -o p1 p1_free_particle.f90 && ./p1
!======================================================================
module numerov_tools
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp), parameter :: pi = 3.14159265358979323_dp
contains
  !----------------------------------------------------------------
  ! Numerov recursion, forward direction, real wavefunction
  ! psi(n+1) c(n+1) = (12 - 10 c(n)) psi(n) - c(n-1) psi(n-1),
  ! c = 1 + h^2 k2 / 12
  !----------------------------------------------------------------
  subroutine numerov(k2, psi, h, n)
    integer,  intent(in)    :: n
    real(dp), intent(in)    :: k2(0:n), h
    real(dp), intent(inout) :: psi(0:n)       ! psi(0), psi(1) given on entry
    real(dp) :: c(0:n)
    integer  :: i
    c = 1.0_dp + h*h*k2/12.0_dp
    do i = 1, n-1
       psi(i+1) = ((12.0_dp - 10.0_dp*c(i))*psi(i) - c(i-1)*psi(i-1)) / c(i+1)
    end do
  end subroutine numerov

  !----------------------------------------------------------------
  ! composite Simpson rule on 0..n (n even)
  !----------------------------------------------------------------
  function simpson(y, h, n) result(s)
    integer,  intent(in) :: n
    real(dp), intent(in) :: y(0:n), h
    real(dp) :: s
    s = h/3.0_dp*( y(0) + y(n) + 4.0_dp*sum(y(1:n-1:2)) + 2.0_dp*sum(y(2:n-2:2)) )
  end function simpson

  !----------------------------------------------------------------
  ! least-squares slope of log(e) vs log(h)  -> observed order
  !----------------------------------------------------------------
  function order(h, e, m) result(p)
    integer,  intent(in) :: m
    real(dp), intent(in) :: h(m), e(m)
    real(dp) :: p, lx(m), ly(m)
    lx = log(h); ly = log(e)
    p = (m*sum(lx*ly) - sum(lx)*sum(ly)) / (m*sum(lx*lx) - sum(lx)**2)
  end function order
end module numerov_tools

module box_problem
```

```
  use numerov_tools
  implicit none
contains
  ! value of psi(L) for trial energy E
  function shoot_box(E, L, n) result(f)
    real(dp), intent(in) :: E, L
    integer,  intent(in) :: n
    real(dp) :: f, h, k2(0:n), psi(0:n)
    h = L/n
    k2 = 2.0_dp*E
    psi(0) = 0.0_dp; psi(1) = 1.0d-3*h
    call numerov(k2, psi, h, n)
    f = psi(n)
  end function shoot_box

  ! n-th eigenvalue + normalised eigenfunction
  subroutine box_eigen(nst, L, n, E, psi)
    integer,  intent(in)  :: nst, n
    real(dp), intent(in)  :: L
    real(dp), intent(out) :: E, psi(0:n)
    real(dp) :: Eex, Ea, Eb, a, b, fa, fb, fm, m, best, h, k2(0:n)
    integer  :: i, nscan = 200
    Eex = (nst*pi/L)**2/2.0_dp
    Ea = 0.5_dp*Eex*(1.0_dp - 1.0_dp/nst) + 1.0d-3
    Eb = Eex*(1.0_dp + 1.0_dp/nst)
    best = huge(1.0_dp)
    fa = shoot_box(Ea, L, n)
    do i = 1, nscan                       ! bracket: sign change nearest Eex
       b  = Ea + (Eb - Ea)*i/nscan
       fb = shoot_box(b, L, n)
       a  = Ea + (Eb - Ea)*(i-1)/nscan
       if (sign(1.0_dp,fa) /= sign(1.0_dp,fb)) then
          if (abs(a - Eex) < best) then
             best = abs(a - Eex); E = a
          end if
       end if
       fa = fb
    end do
    a = E; b = E + (Eb - Ea)/nscan; fa = shoot_box(a, L, n)
    do while (b - a > 1.0d-13*max(1.0_dp, abs(a)))      ! bisection
       m = 0.5_dp*(a + b); fm = shoot_box(m, L, n)
       if (sign(1.0_dp,fm) == sign(1.0_dp,fa)) then
          a = m; fa = fm
       else
          b = m
       end if
    end do
    E = 0.5_dp*(a + b)
    h = L/n; k2 = 2.0_dp*E
    psi(0) = 0.0_dp; psi(1) = 1.0d-3*h
    call numerov(k2, psi, h, n)
    psi = psi/sqrt(simpson(psi**2, h, n))               ! normalisation
    if (sum(psi*sin(nst*pi*[(i*h, i=0,n)]/L)) < 0.0_dp) psi = -psi
  end subroutine box_eigen
end module box_problem

program free_particle
  use numerov_tools
  use box_problem
  implicit none
  integer, parameter :: nA = 8, nB = 7
  integer  :: NsA(nA) = [25, 50, 100, 200, 400, 800, 1600, 3200]
  integer  :: NsB(nB) = [20, 40, 80, 160, 320, 640, 1280]
  real(dp) :: hA(nA), eA(nA), eT(nA), hB(nB), eB(3, nB)
  real(dp), allocatable :: x(:), psi(:), k2(:), ex(:), st(:,:)
  real(dp) :: k, X0, h, E, Eex, Eh, Eh2, ER
  integer  :: i, j, ng, s
  character(len=*), parameter :: fmtA = '(i6,f10.5,es18.3,f8.2,es18.3,f8.2)'

  !------------------------------ PART A
  k = 2.0_dp; X0 = 10.0_dp
  print '(a)', repeat('=',70)
  print '(a)', 'PART A : free particle, psi(0)=0, k = 2, exact solution sin(kx)'
  print '(a)', repeat('=',70)
  print '(a6,a10,a18,a8,a18,a8)', 'N', 'h', 'err(exact start)', 'ratio', 'err(Taylor start)', 'ratio'
  open(10, file='p1_partA.dat', status='replace')
  do i = 1, nA
     ng = NsA(i); h = X0/ng
     allocate(x(0:ng), psi(0:ng), k2(0:ng))
     x = [(j*h, j=0,ng)]; k2 = k*k
     psi(0) = 0.0_dp; psi(1) = sin(k*h)        ! exact start
     call numerov(k2, psi, h, ng)
```

```
    eA(i) = maxval(abs(psi - sin(k*x)))
    psi(0) = 0.0_dp; psi(1) = k*h             ! Taylor start
    call numerov(k2, psi, h, ng)
    eT(i) = maxval(abs(psi - sin(k*x)))
    hA(i) = h
    if (i == 1) then
       print fmtA, ng, h, eA(i), 0.0, eT(i), 0.0
    else
       print fmtA, ng, h, eA(i), eA(i-1)/eA(i), eT(i), eT(i-1)/eT(i)
    end if
    write(10,'(3es16.7)') h, eA(i), eT(i)
    deallocate(x, psi, k2)
  end do
  close(10)
  print '(a,f6.2,a)', 'observed order (exact start)  p = ', order(hA, eA, nA), '  (theory 4)'
  print '(a,f6.2,a)', 'observed order (Taylor start) p = ', order(hA, eT, nA), '  (theory 2)'

  !------------------------------ PART B
  print '(/a)', repeat('=',70)
  print '(a)', 'PART B : free particle in a box L = 1  (E_n = n^2 pi^2 / 2)'
  print '(a)', repeat('=',70)
  print '(a3,a18,a18,a11,a16,a15)', 'n', 'E_numerov', 'E_exact', 'rel.err', 'norm', 'max|psi-ex|'
  ng = 200; h = 1.0_dp/ng
  allocate(x(0:ng), psi(0:ng), ex(0:ng), st(0:ng,4))
  x = [(j*h, j=0,ng)]
  do s = 1, 4
     call box_eigen(s, 1.0_dp, ng, E, psi)
     Eex = (s*pi)**2/2.0_dp
     ex  = sqrt(2.0_dp)*sin(s*pi*x)
     st(:,s) = psi
     print '(i3,2f18.12,es11.3,f16.12,es15.3)', s, E, Eex, abs(E-Eex)/Eex, &
          simpson(psi**2,h,ng), maxval(abs(psi-ex))
  end do
  print '(a)', 'overlap matrix <psi_i|psi_j>  (should be identity):'
  do i = 1, 4
     write(*,'(4es11.2)') (simpson(st(:,i)*st(:,j),h,ng), j=1,4)
  end do
  open(11, file='p1_states.dat', status='replace')
  do j = 0, ng
     write(11,'(5es16.7)') x(j), st(j,1:4)
  end do
  close(11)
  deallocate(x, psi, ex, st)

  print '(/a)', 'Convergence of the eigenvalues in the box:'
  open(12, file='p1_partB.dat', status='replace')
  do i = 1, nB
     ng = NsB(i); hB(i) = 1.0_dp/ng
     allocate(psi(0:ng))
     do s = 1, 3
        call box_eigen(s, 1.0_dp, ng, E, psi)
        Eex = (s*pi)**2/2.0_dp
        eB(s,i) = abs(E-Eex)/Eex
     end do
     write(12,'(4es16.7)') hB(i), eB(:,i)
     deallocate(psi)
  end do
  close(12)
  do s = 1, 3
     write(*,'(a,i1,a,7es10.2,a,f5.2)') '  n=', s, ': ', eB(s,:), ' -> order', order(hB(1:5), eB(s,1:5), 5)
  end do

  ! Richardson extrapolation (error ~ h^4 -> weight 16/15, -1/15)
  allocate(psi(0:80))
  call box_eigen(1, 1.0_dp, 40, Eh, psi(0:40))
  call box_eigen(1, 1.0_dp, 80, Eh2, psi)
  ER = (16.0_dp*Eh2 - Eh)/15.0_dp
  print '(/a,f15.12,a,f15.12,a,f15.12,a,es9.2)', 'Richardson n=1: E(h)=', Eh, ' E(h/2)=', Eh2, &
       ' E_R=', ER, ' err=', abs(ER - pi**2/2)
end program free_particle
```

Results, normalisation and verification

Part A — accuracy of the integration ($k = 2$, $X = 10$, i.e. about 3 wavelengths).

N	h	error (exact start)	ratio	error (Taylor start)	ratio
25	0.400	1.55e-02	—	1.14e-01	—
50	0.200	1.01e-03	15.4	2.71e-02	4.2

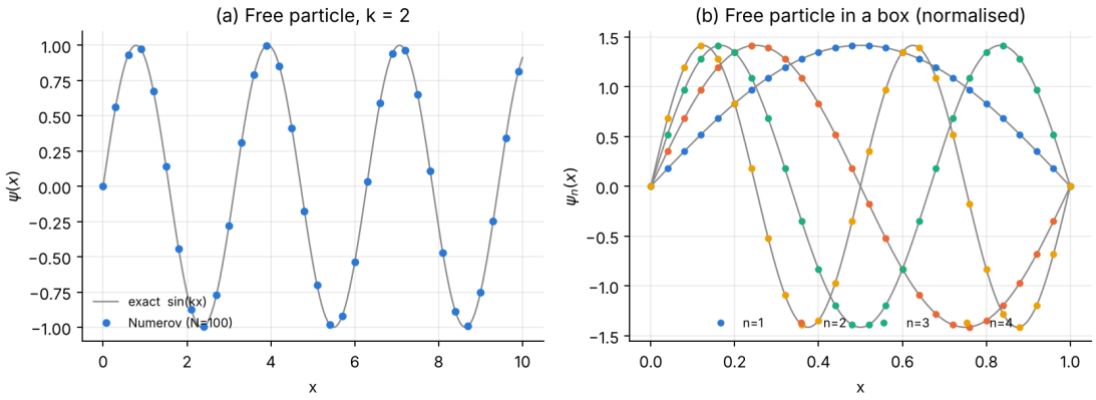
N	h	error (exact start)	ratio	error (Taylor start)	ratio
100	0.100	6.29e-05	16.1	6.69e-03	4.1
200	0.050	3.92e-06	16.0	1.67e-03	4.0
400	0.025	2.46e-07	16.0	4.17e-04	4.0
800	0.0125	1.53e-08	16.0	1.04e-04	4.0
1600	0.00625	9.50e-10	16.1	2.60e-05	4.0
3200	0.00313	5.40e-11	17.6	6.51e-06	4.0

Fitted orders: $p = 4.01$ with the exact start, $p = 2.01$ with the Taylor start — exactly as predicted by the starting-value analysis of Part I (fault no. 2).

Part B — box of length $L = 1$, $N = 200$ ($h = 0.005$).

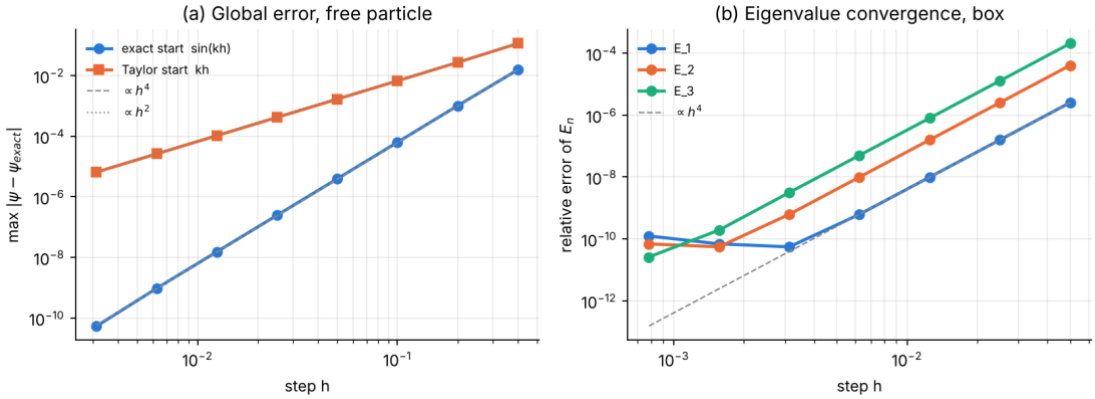
n	E_n Numerov	E_n exact	rel. error	$\int\psi^2 dx$	$\max\lvert\psi - \psi_{ex}\rvert$
1	4.934802199274	4.934802200545	2.6e-10	1.000000000000	1.6e-11
2	19.739208722038	19.739208802179	4.1e-09	1.000000000000	1.3e-11
3	44.413218892273	44.413219804902	2.1e-08	1.000000000000	5.4e-12
4	78.956830080506	78.956835208715	6.5e-08	1.000000000000	4.3e-12

The overlap matrix $\langle\psi_i|\psi_j\rangle$ equals the identity to within 5×10^{-12}: the numerical eigenfunctions are normalised and mutually orthogonal, as eigenfunctions of a Hermitian operator must be. The relative energy error grows like n^4 because the error constant contains k^4h^4 — higher states have shorter wavelengths and are resolved by fewer points.



Problem 1. (a) Numerov solution of the free particle compared with $\sin kx$. (b) The first four normalised eigenfunctions in the box (dots) on top of the exact ones (grey).

Error analysis and convergence



Problem 1, convergence. (a) Global error of the free-particle integration: slope 4 with the exact start, slope 2 with the Taylor start. (b) Relative error of E_1, E_2, E_3 in the box: slope 4 until the round-off floor near 10^{-10} is reached.

Observations:

- The eigenvalue errors fall with fitted orders 3.90, 4.00, 4.00 for $n = 1,2,3$.
- For E_1 the error stops decreasing below $h \approx 3 \times 10^{-3}$ and then grows slightly: this is the **round-off floor** (fault no. 8).
- **Richardson extrapolation** from $N = 40$ and $N = 80$: $E(h) = 4.934801417973$, $E(h/2) = 4.934802151642$ give $E_R = 4.934802200554$, error 9×10^{-12} — better than a direct calculation with 16 times more points.

Part III — Problem 2: The square potential barrier

Physics and analytical solution

$$V(x) = \begin{cases} V_0, & 0 < x < a \\ 0, & \text{otherwise} \end{cases} \qquad V_0 = 5,\ a = 1.$$

A particle of energy $E > 0$ arrives from the left. The solution in the three regions is

$$\psi(x) = \begin{cases} Ae^{ikx} + Be^{-ikx}, & x < 0 \quad \text{(incident + reflected)} \\ Ce^{\kappa x} + De^{-\kappa x}, & 0 < x < a \quad \left(E < V_0,\ \kappa = \sqrt{2(V_0 - E)}\right) \\ Fe^{ikx}, & x > a \quad \text{(transmitted only)} \end{cases} \qquad k = \sqrt{2E}.$$

For $E > V_0$ replace $\kappa \to iq$ with $q = \sqrt{2(E - V_0)}$. Continuity of ψ and ψ' at $x = 0$ and $x = a$ gives four linear equations; eliminating B, C, D yields the transmission and reflection coefficients

$$T = \left|\frac{F}{A}\right|^2 = \begin{cases} \left[1 + \dfrac{V_0^2 \sinh^2(\kappa a)}{4E(V_0 - E)}\right]^{-1}, & E < V_0 \quad \text{(tunnelling)} \\ \left[1 + \dfrac{V_0^2 \sin^2(qa)}{4E(E - V_0)}\right]^{-1}, & E > V_0 \end{cases} \qquad R = \left|\frac{B}{A}\right|^2 = 1 - T.$$

At $E = V_0$ both formulas tend to $T = (1 + V_0 a^2/2)^{-1}$. Above the barrier, **transmission resonances** $T = 1$ occur when $qa = n\pi$, i.e. $E = V_0 + n^2\pi^2/2a^2$; the first one is $E = 9.9348$.

Normalisation of a scattering state. ψ is not square-integrable, so the normalisation is fixed by the *incident amplitude* ($A = 1$) and the meaningful check is conservation of the probability current

$$j = \frac{1}{2i}(\psi^*\psi' - \psi\psi^{*\prime}), \qquad j_{\text{inc}} = j_{\text{refl}} + j_{\text{trans}} \ \Leftrightarrow\ R + T = 1.$$

Numerical scheme

- **Backward integration.** Because the solution on the right contains only one wave, we fix $F = 1$ and start there:

  $$\psi_N = e^{ikx_N}, \qquad \psi_{N-1} = e^{ikx_{N-1}} \quad \text{(exact starting values)}.$$

  The recursion is run backwards with **complex** arithmetic,

  $$\psi_{j-1} = \frac{\left(12 - 10c_j\right)\psi_j - c_{j+1}\psi_{j+1}}{c_{j-1}}, \qquad j = N - 1, \ldots, 1.$$

  No energy search is needed: every $E > 0$ is allowed (continuous spectrum).

- **Extracting A and B.** On the two leftmost nodes, which lie in the free region,

  $$\begin{pmatrix} e^{ikx_0} & e^{-ikx_0} \\ e^{ikx_1} & e^{-ikx_1} \end{pmatrix} \begin{pmatrix} A \\ B \end{pmatrix} = \begin{pmatrix} \psi_0 \\ \psi_1 \end{pmatrix}, \qquad T = \frac{1}{|A|^2}, \quad R = \frac{|B|^2}{|A|^2}.$$

- **The discontinuity (fault no. 3).** Numerov's $O(h^4)$ accuracy assumes that $\psi^{(6)}$ exists. At a step in V, ψ'' jumps and ψ''' jumps by $-[k^2]\psi'$. Three treatments are compared:

  1. *misaligned grid* — the edges fall inside a cell;
  2. *aligned grid* — the edges are grid nodes and V there is set to the mean value $V_0/2$;

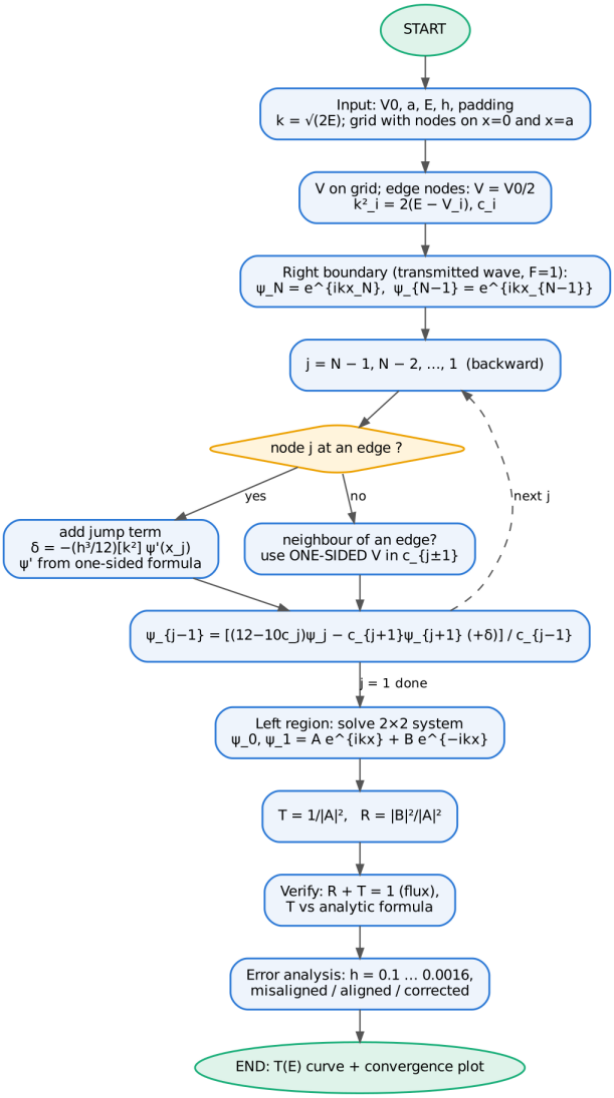
3. *corrected scheme* — aligned grid, plus (a) every stencil centred **off** an edge uses the one-sided value of V at a neighbouring edge node (the value on its own side), and (b) the stencil centred **on** an edge node x_e receives the jump correction derived by repeating the Taylor expansion of Part I with one-sided derivatives:

$$\delta = -\frac{h^3}{12}\,[k^2]_e\,\psi'(x_e), \qquad [k^2]_e = k^2(x_e^+) - k^2(x_e^-),$$

added to the right-hand side of the recursion. $\psi'(x_e)$ is needed only to $O(h^2)$ and is taken from the already computed side: $\psi'(x_e) \approx (-3\psi_e + 4\psi_{e+1} - \psi_{e+2})/2h$.

Derivation of (b) in one line: with one-sided expansions, $\psi_{e+1} + \psi_{e-1} - 2\psi_e$ contains the term $\frac{h^3}{6}(\psi'''_+ - \psi'''_-)$, whereas the Numerov combination with $\bar{V} = V_0/2$ reproduces only $\frac{h^3}{12}(\psi'''_+ - \psi'''_-)$. The difference $\frac{h^3}{12}[\psi'''] = -\frac{h^3}{12}[k^2]\psi'$ is a local $O(h^3)$ error which, by the argument of Part I, would give a global $O(h^2)$ error unless it is added back.

Logic chart



Logic chart of the barrier program (corrected scheme).

Python 3 code

File: `p2_square_barrier.py`

```
#!/usr/bin/env python3
"""
Problem 2 -- RECTANGULAR (SQUARE) POTENTIAL BARRIER with the Numerov method
============================================================================
Units: hbar = m = 1.  V(x) = V0 for 0 < x < a, 0 elsewhere.
psi''(x) = -k^2(x) psi(x),  k^2(x) = 2 [E - V(x)]   (k^2 < 0 inside barrier if E < V0)

Scattering boundary conditions (wave incident from the left):
    x < 0 :  psi = A e^{ikx} + B e^{-ikx}      (incident + reflected)
    x > a :  psi = F e^{ikx}                    (transmitted only)
Numerical strategy ("backward integration"):
    choose F = 1, impose psi = e^{ikx} at the two right-most grid points,
    integrate the COMPLEX Numerov recursion from right to left,
    at the two left-most grid points decompose psi into A, B.
    T = |F/A|^2 = 1/|A|^2,   R = |B/A|^2.
Verification ("normalisation" of a scattering state = probability-flux
conservation):  R + T = 1.   Accuracy: compare T with the analytic formula.

Discontinuity issue: V jumps at x = 0 and x = a.  Numerov's O(h^4) error
assumes a smooth psi''.  We study two grids:
    (i)  edges ON grid nodes, with V(edge) = V0/2 (mean value)  -> O(h^4)
    (ii) edges falling INSIDE a cell (misaligned grid)            -> O(h^2) or worse
Outputs: tables + p2_wavefunction.png, p2_transmission.png, p2_convergence.png
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4"]
V0, A_WIDTH = 5.0, 1.0           # barrier height and width


def T_exact(E, V0=V0, a=A_WIDTH):
    """Analytic transmission coefficient of a rectangular barrier."""
    if abs(E - V0) < 1e-12:
        return 1.0 / (1.0 + V0 * a * a / 2.0)          # limit E -> V0  (m = hbar = 1)
    if E < V0:
        kap = np.sqrt(2.0 * (V0 - E))
        return 1.0 / (1.0 + V0 ** 2 * np.sinh(kap * a) ** 2 / (4.0 * E * (V0 - E)))
    q = np.sqrt(2.0 * (E - V0))
    return 1.0 / (1.0 + V0 ** 2 * np.sin(q * a) ** 2 / (4.0 * E * (E - V0)))


def potential(x, h, aligned):
    V = np.where((x > 0.0) & (x < A_WIDTH), V0, 0.0)
    edges = []
    if aligned:                                          # mean value on the edge nodes
        for xe in (0.0, A_WIDTH):
            i = np.where(np.isclose(x, xe, atol=1e-3 * h))[0]
            if len(i):
                V[i[0]] = V0 / 2
                edges.append(i[0])
    return V, edges


def scatter(E, h, mode="corrected", Lpad=2.0, return_wave=False):
    """mode = 'misaligned' : barrier edges fall inside grid cells
              'aligned'    : edges on nodes, V(edge) = V0/2
              'corrected'  : aligned + jump correction term delta at edge nodes"""
    k = np.sqrt(2.0 * E)
    aligned = mode != "misaligned"
    shift = 0.0 if aligned else 0.37 * h                 # edges inside cells if shifted
    xl, xr = -Lpad + shift, A_WIDTH + Lpad + shift
    N = int(round((xr - xl) / h))
    x = xl + h * np.arange(N + 1)
    V, edges = potential(x, h, aligned)
    k2 = 2.0 * (E - V)
    c = 1.0 + h * h * k2 / 12.0
    is_edge = np.zeros(N + 1, dtype=bool)
    is_edge[edges] = True
    jump = {}
    if mode == "corrected":
        for i in edges:   # jump of k^2 across the edge node: right limit - left limit
            jump[i] = 2.0 * (E - V[i + 1]) - 2.0 * (E - V[i - 1])
    psi = np.zeros(N + 1, dtype=complex)
    psi[N], psi[N - 1] = np.exp(1j * k * x[N]), np.exp(1j * k * x[N - 1])   # F = 1
```

```
    for n in range(N - 1, 0, -1):                       # backward Numerov
        cp, cm = c[n + 1], c[n - 1]
        if mode == "corrected" and not is_edge[n]:
            # a stencil centred OFF the edge must see the ONE-SIDED value of V
            # at a neighbouring edge node (the value on its own side), not V0/2
            if is_edge[n + 1]:
                cp = 1.0 + h * h * 2.0 * (E - V[n]) / 12.0
            if is_edge[n - 1]:
                cm = 1.0 + h * h * 2.0 * (E - V[n]) / 12.0
        rhs = (12.0 - 10.0 * c[n]) * psi[n] - cp * psi[n + 1]
        if n in jump:
            # stencil centred ON the edge: V0/2 at the centre + jump term
            # delta = -h^3/12 [k^2] psi'(x_n), psi' from the known right side (O(h^2))
            dpsi = (-3.0 * psi[n] + 4.0 * psi[n + 1] - psi[n + 2]) / (2.0 * h)
            rhs += -h ** 3 / 12.0 * jump[n] * dpsi
        psi[n - 1] = rhs / cm
    # decomposition  psi = A e^{ikx} + B e^{-ikx}  at x0, x1
    M = np.array([[np.exp(1j * k * x[0]), np.exp(-1j * k * x[0])],
                  [np.exp(1j * k * x[1]), np.exp(-1j * k * x[1])]])
    Acoef, Bcoef = np.linalg.solve(M, psi[:2])
    T = 1.0 / abs(Acoef) ** 2
    R = abs(Bcoef) ** 2 / abs(Acoef) ** 2
    if return_wave:
        return T, R, x, psi / Acoef                     # incident amplitude -> 1
    return T, R


def slope(h, e):
    return np.polyfit(np.log(h), np.log(e), 1)[0]


def main():
    print("=" * 74)
    print(f"Square barrier  V0 = {V0},  a = {A_WIDTH}   (hbar = m = 1)")
    print("=" * 74)
    h = 0.01
    print(f"{'E':>6} {'T_numerov':>14} {'T_exact':>14} {'|dT|':>10} {'R':>12} {'R+T-1':>11}")
    for E in (0.5, 1.0, 2.5, 4.0, 4.9, 5.1, 6.0, 8.0, 9.935, 12.0, 15.0):
        T, R = scatter(E, h)
        print(f"{E:6.3f} {T:14.10f} {T_exact(E):14.10f} {abs(T-T_exact(E)):10.2e} "
              f"{R:12.8f} {R+T-1:11.2e}")

    # ------------------------------------------------ convergence study
    hs = np.array([0.1, 0.05, 0.025, 0.0125, 0.00625, 0.003125, 0.0015625])
    Es = (2.0, 8.0)
    modes = ("misaligned", "aligned", "corrected")
    err = {(E, m): [] for E in Es for m in modes}
    flux = {(E, m): [] for E in Es for m in modes}
    for E in Es:
        for m in modes:
            for hh in hs:
                T, R = scatter(E, hh, mode=m)
                err[E, m].append(abs(T - T_exact(E)))
                flux[E, m].append(abs(R + T - 1.0))
    for E in Es:
        print(f"\nConvergence of T at E = {E}   (T_exact = {T_exact(E):.12f})")
        print(f"{'h':>10} {'misaligned':>12} {'aligned V0/2':>13} {'corrected':>12} {'|R+T-1| corr.':>14}")
        for i, hh in enumerate(hs):
            print(f"{hh:10.6f} {err[E,'misaligned'][i]:12.2e} {err[E,'aligned'][i]:13.2e} "
                  f"{err[E,'corrected'][i]:12.2e} {flux[E,'corrected'][i]:14.1e}")
        print("observed order: " + ", ".join(f"{m} = {slope(hs[:5], err[E, m][:5]):.2f}" for m in modes))

    # ------------------------------------------------ figures
    fig, ax = plt.subplots(figsize=(8, 3.6))
    for i, E in enumerate((2.0, 8.0)):
        T, R, x, psi = scatter(E, 0.005, return_wave=True)
        ax.plot(x, abs(psi) ** 2, color=C[i], lw=2, label=f"E = {E}  (T = {T:.4f})")
    ax.axvspan(0, A_WIDTH, color="#cccccc", alpha=.5, lw=0)
    ax.text(A_WIDTH / 2, ax.get_ylim()[1] * .92, f"barrier V0={V0}", ha="center", fontsize=8)
    ax.set(xlabel="x", ylabel=r"$|\psi(x)|^2$  (incident amplitude 1)",
           title="Probability density across the barrier")
    ax.legend(frameon=False, fontsize=8, loc="upper right")
    ax.spines[["top", "right"]].set_visible(False); ax.grid(alpha=.25)
    fig.tight_layout(); fig.savefig("p2_wavefunction.png", dpi=170)

    Eg = np.linspace(0.05, 20, 400)
    Tn = np.array([scatter(E, 0.01)[0] for E in Eg])
    Te = np.array([T_exact(E) for E in Eg])
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    ax[0].plot(Eg, Te, color="#888888", lw=1, label="analytic")
    ax[0].plot(Eg[::8], Tn[::8], "o", ms=4, color=C[0], label="Numerov h=0.01")
    ax[0].axvline(V0, color="#888888", ls=":", lw=1)
```

```
    ax[0].set(xlabel="E", ylabel="T(E)", title="(a) Transmission coefficient")
    ax[0].legend(frameon=False, fontsize=8, loc="lower right")
    ax[1].semilogy(Eg, abs(Tn - Te) + 1e-18, color=C[1], lw=1.5)
    ax[1].set(xlabel="E", ylabel=r"$|T_{num}-T_{exact}|$", title="(b) Absolute error, h = 0.01")
    for a in ax:
        a.spines[["top", "right"]].set_visible(False); a.grid(alpha=.25)
    fig.tight_layout(); fig.savefig("p2_transmission.png", dpi=170)

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.9), sharey=True)
    styles = {"misaligned": ("s--", C[1]), "aligned": ("^-.", C[3]), "corrected": ("o-", C[0])}
    labels = {"misaligned": "edges inside cells", "aligned": "edges on nodes, V=V0/2",
              "corrected": "edges on nodes + jump correction"}
    for j, E in enumerate(Es):
        for m in modes:
            st, col = styles[m]
            ax[j].loglog(hs, err[E, m], st, color=col, lw=2, ms=5, label=labels[m])
        e0 = err[E, "corrected"][0]
        ax[j].loglog(hs, e0 * (hs / hs[0]) ** 4, ":", color="#888888", lw=1, label=r"$\propto h^4$")
        ax[j].loglog(hs, err[E, "aligned"][0] * (hs / hs[0]) ** 2, "-", color="#bbbbbb", lw=1, label=r"$\propto
h^2$")
        ax[j].set(xlabel="step h", title=f"E = {E}" + ("  (tunnelling)" if E < V0 else "  (above barrier)"))
        ax[j].spines[["top", "right"]].set_visible(False); ax[j].grid(alpha=.25, which="both")
    ax[0].set_ylabel(r"$|T_{num}-T_{exact}|$")
    ax[0].legend(frameon=False, fontsize=7, loc="lower right")
    fig.tight_layout(); fig.savefig("p2_convergence.png", dpi=170)
    print("\nfigures written: p2_wavefunction.png, p2_transmission.png, p2_convergence.png")


if __name__ == "__main__":
    main()
```

Fortran 90 code

File: `p2_square_barrier.f90`

```
!=======================================================================
! Problem 2 -- RECTANGULAR POTENTIAL BARRIER with the Numerov method
! Units hbar = m = 1.  V = V0 for 0 < x < a, 0 elsewhere.
! psi'' = -k^2(x) psi,  k^2 = 2(E - V)
!
! Backward integration of the COMPLEX Numerov recursion from the right,
! where psi = exp(ikx) (transmitted wave, F = 1), to the left, where
! psi = A exp(ikx) + B exp(-ikx).   T = 1/|A|^2,  R = |B|^2/|A|^2.
! Checks: R + T = 1 (flux conservation) and T vs analytic formula.
! Three treatments of the potential discontinuity (mode):
!   1 = misaligned : barrier edges inside grid cells
!   2 = aligned    : edges on grid nodes, V(edge) = V0/2
!   3 = corrected  : aligned + one-sided V in neighbouring stencils
!                    + jump term delta = -h^3/12 [k^2] psi'(edge)
! Output : screen tables + p2_T.dat, p2_conv.dat, p2_wave.dat
! Compile: gfortran -O2 -o p2 p2_square_barrier.f90 && ./p2
!=======================================================================
module barrier
  implicit none
  integer,  parameter :: dp = kind(1.0d0)
  real(dp), parameter :: V0 = 5.0_dp, aw = 1.0_dp, Lpad = 2.0_dp
  complex(dp), parameter :: iu = (0.0_dp, 1.0_dp)
contains
  function T_exact(E) result(T)
    real(dp), intent(in) :: E
    real(dp) :: T, kap, q
    if (abs(E - V0) < 1.0d-12) then
       T = 1.0_dp/(1.0_dp + V0*aw*aw/2.0_dp)
    else if (E < V0) then
       kap = sqrt(2.0_dp*(V0 - E))
       T = 1.0_dp/(1.0_dp + V0**2*sinh(kap*aw)**2/(4.0_dp*E*(V0 - E)))
    else
       q = sqrt(2.0_dp*(E - V0))
       T = 1.0_dp/(1.0_dp + V0**2*sin(q*aw)**2/(4.0_dp*E*(E - V0)))
    end if
  end function T_exact

  subroutine scatter(E, h, mode, T, R, wavefile)
    real(dp), intent(in)  :: E, h
    integer,  intent(in)  :: mode
    real(dp), intent(out) :: T, R
    character(len=*), intent(in), optional :: wavefile
    real(dp), allocatable :: x(:), Vx(:), k2(:), c(:), jump(:)
    logical,  allocatable :: edge(:)
    complex(dp), allocatable :: psi(:)
```

```
    complex(dp) :: rhs, dpsi, M(2,2), det, Ac, Bc
    real(dp) :: k, xl, xr, shift, cp, cm
    integer  :: N, i, j

    k = sqrt(2.0_dp*E)
    shift = 0.0_dp
    if (mode == 1) shift = 0.37_dp*h
    xl = -Lpad + shift; xr = aw + Lpad + shift
    N = nint((xr - xl)/h)
    allocate(x(0:N), Vx(0:N), k2(0:N), c(0:N), psi(0:N), edge(0:N), jump(0:N))
    x = [(xl + i*h, i=0,N)]
    Vx = 0.0_dp; edge = .false.; jump = 0.0_dp
    where (x > 0.0_dp .and. x < aw) Vx = V0
    if (mode /= 1) then                       ! edge nodes get the mean value
       do i = 0, N
          if (abs(x(i)) < 1.0d-3*h .or. abs(x(i) - aw) < 1.0d-3*h) then
             Vx(i) = V0/2.0_dp; edge(i) = .true.
          end if
       end do
    end if
    k2 = 2.0_dp*(E - Vx)
    c  = 1.0_dp + h*h*k2/12.0_dp
    if (mode == 3) then
       do i = 1, N-1
          if (edge(i)) jump(i) = 2.0_dp*(E - Vx(i+1)) - 2.0_dp*(E - Vx(i-1))
       end do
    end if

    psi(N)   = exp(iu*k*x(N))                 ! transmitted wave, F = 1
    psi(N-1) = exp(iu*k*x(N-1))
    do j = N-1, 1, -1                         ! backward Numerov
       cp = c(j+1); cm = c(j-1)
       if (mode == 3 .and. .not. edge(j)) then
          if (edge(j+1)) cp = 1.0_dp + h*h*2.0_dp*(E - Vx(j))/12.0_dp
          if (edge(j-1)) cm = 1.0_dp + h*h*2.0_dp*(E - Vx(j))/12.0_dp
       end if
       rhs = (12.0_dp - 10.0_dp*c(j))*psi(j) - cp*psi(j+1)
       if (mode == 3 .and. edge(j)) then
          dpsi = (-3.0_dp*psi(j) + 4.0_dp*psi(j+1) - psi(j+2))/(2.0_dp*h)
          rhs  = rhs - h**3/12.0_dp*jump(j)*dpsi
       end if
       psi(j-1) = rhs/cm
    end do

    ! solve [e^{ikx0} e^{-ikx0}; e^{ikx1} e^{-ikx1}] (A,B) = (psi0, psi1)
    M(1,1) = exp(iu*k*x(0)); M(1,2) = exp(-iu*k*x(0))
    M(2,1) = exp(iu*k*x(1)); M(2,2) = exp(-iu*k*x(1))
    det = M(1,1)*M(2,2) - M(1,2)*M(2,1)
    Ac  = ( M(2,2)*psi(0) - M(1,2)*psi(1))/det
    Bc  = (-M(2,1)*psi(0) + M(1,1)*psi(1))/det
    T = 1.0_dp/abs(Ac)**2
    R = abs(Bc)**2/abs(Ac)**2

    if (present(wavefile)) then
       open(20, file=wavefile, status='replace')
       do i = 0, N
          write(20,'(2es16.7)') x(i), abs(psi(i)/Ac)**2
       end do
       close(20)
    end if
    deallocate(x, Vx, k2, c, psi, edge, jump)
  end subroutine scatter

  function order(h, e, m) result(p)
    integer,  intent(in) :: m
    real(dp), intent(in) :: h(m), e(m)
    real(dp) :: p, lx(m), ly(m)
    lx = log(h); ly = log(e)
    p = (m*sum(lx*ly) - sum(lx)*sum(ly)) / (m*sum(lx*lx) - sum(lx)**2)
  end function order
end module barrier

program square_barrier
  use barrier
  implicit none
  real(dp) :: Elist(11) = [0.5_dp, 1.0_dp, 2.5_dp, 4.0_dp, 4.9_dp, 5.1_dp, 6.0_dp, &
                           8.0_dp, 9.935_dp, 12.0_dp, 15.0_dp]
  real(dp) :: hs(7) = [0.1_dp, 0.05_dp, 0.025_dp, 0.0125_dp, 0.00625_dp, 0.003125_dp, 0.0015625_dp]
  real(dp) :: Es(2) = [2.0_dp, 8.0_dp]
  real(dp) :: err(3,7), fl(7), T, R, E
  integer  :: i, ie, md
```

```
  print '(a)', repeat('=',74)
  print '(a)', 'Square barrier  V0 = 5,  a = 1   (hbar = m = 1),  h = 0.01, corrected scheme'
  print '(a)', repeat('=',74)
  print '(a6,2a15,a11,a13,a12)', 'E', 'T_numerov', 'T_exact', '|dT|', 'R', 'R+T-1'
  do i = 1, 11
     call scatter(Elist(i), 0.01_dp, 3, T, R)
     print '(f6.3,2f15.10,es11.2,f13.8,es12.2)', Elist(i), T, T_exact(Elist(i)), &
          abs(T - T_exact(Elist(i))), R, R + T - 1.0_dp
  end do

  open(11, file='p2_T.dat', status='replace')
  do i = 1, 400
     E = 0.05_dp + (20.0_dp - 0.05_dp)*(i-1)/399.0_dp
     call scatter(E, 0.01_dp, 3, T, R)
     write(11,'(3es16.7)') E, T, T_exact(E)
  end do
  close(11)

  open(12, file='p2_conv.dat', status='replace')
  do ie = 1, 2
     do i = 1, 7
        do md = 1, 3
           call scatter(Es(ie), hs(i), md, T, R)
           err(md,i) = abs(T - T_exact(Es(ie)))
           if (md == 3) fl(i) = abs(R + T - 1.0_dp)
        end do
        write(12,'(f6.2,5es16.7)') Es(ie), hs(i), err(:,i), fl(i)
     end do
     print '(/a,f4.1,a,f15.12)', 'Convergence of T at E = ', Es(ie), '   T_exact = ', T_exact(Es(ie))
     print '(a10,a13,a14,a13,a15)', 'h', 'misaligned', 'aligned V0/2', 'corrected', '|R+T-1| corr.'
     do i = 1, 7
        print '(f10.6,es13.2,es14.2,es13.2,es15.1)', hs(i), err(:,i), fl(i)
     end do
     print '(a,3f6.2)', 'observed order (misaligned, aligned, corrected): ', &
          order(hs(1:5), err(1,1:5), 5), order(hs(1:5), err(2,1:5), 5), order(hs(1:5), err(3,1:5), 5)
  end do
  close(12)

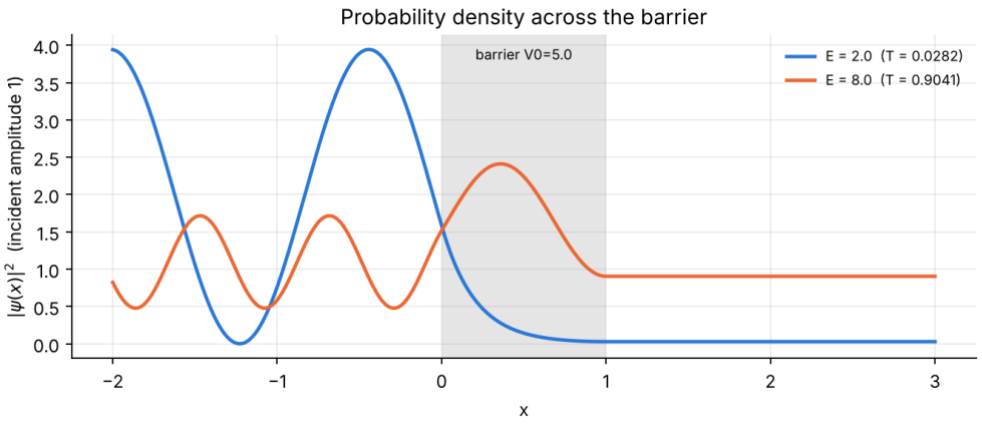
  call scatter(2.0_dp, 0.005_dp, 3, T, R, 'p2_wave_E2.dat')
  call scatter(8.0_dp, 0.005_dp, 3, T, R, 'p2_wave_E8.dat')
end program square_barrier
```

Results, normalisation and verification

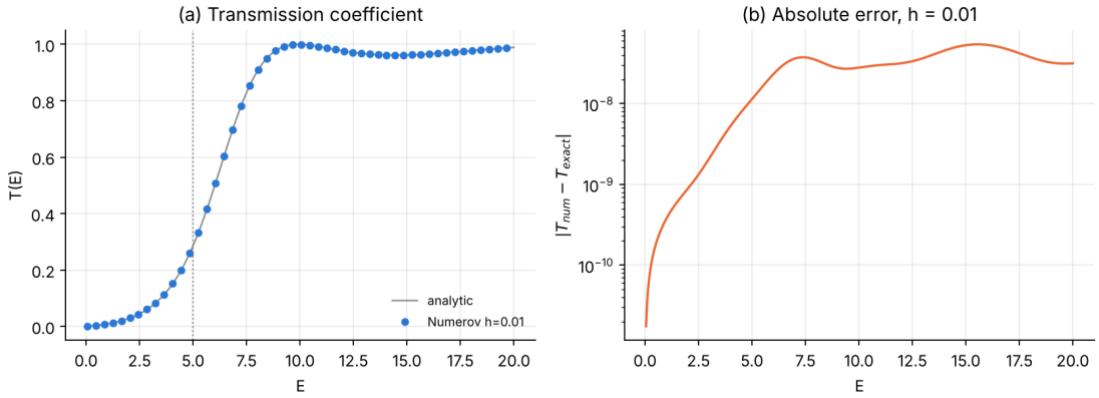
Transmission for $V_0 = 5$, $a = 1$, $h = 0.01$, corrected scheme:

E	T Numerov	T exact	|ΔT|	R	R + T − 1
0.5	0.0035743425	0.0035743427	1.8e-10	0.99642566	-9.9e-11
1.0	0.0089257613	0.0089257617	3.8e-10	0.99107424	-2.5e-10
2.5	0.0446653204	0.0446653217	1.3e-09	0.95533468	-1.2e-09
4.0	0.1459692940	0.1459692993	5.3e-09	0.85403070	-4.1e-09
4.9	0.2684067226	0.2684067332	1.1e-08	0.73159327	-7.5e-09
5.1	0.3037780173	0.3037780296	1.2e-08	0.69622197	-8.4e-09
6.0	0.4959493199	0.4959493430	2.3e-08	0.50405067	-1.4e-08
8.0	0.9041154211	0.9041154557	3.5e-08	0.09588455	-2.5e-08
9.935	0.9999999717	0.9999999995	2.8e-08	0.00000000	-2.8e-08
12.0	0.9768235598	0.9768235914	3.2e-08	0.02317641	-2.7e-08
15.0	0.9621790687	0.9621791216	5.3e-08	0.03782090	-2.7e-08

Tunnelling ($E < V_0$) gives small but finite T; above the barrier T oscillates and reaches 1 at the resonance $E \approx 9.935$.



Problem 2. Probability density for an incident wave of unit amplitude. For $E = 2$ (tunnelling) the standing-wave pattern on the left shows strong reflection and $|\psi|^2$ decays exponentially inside the barrier; for $E = 8$ the wave passes almost freely. On the right $|\psi|^2 = T$ is constant (pure travelling wave).

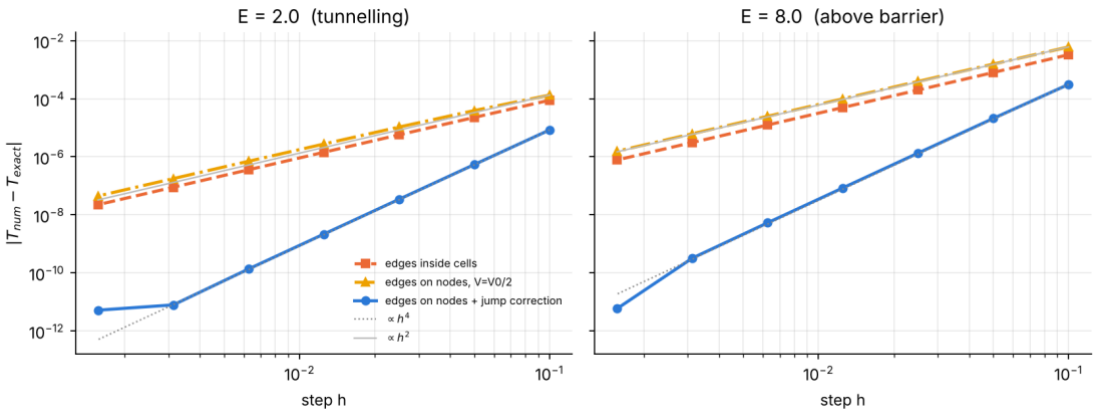


Problem 2. (a) Numerical and analytical transmission coefficients. (b) Absolute error over the whole energy range at $h = 0.01$.

An important lesson about the flux check. With the *uncorrected* schemes, $R + T - 1$ came out at the 10^{-14} level for every h, although T itself was wrong by up to 10^{-4} (e.g. aligned grid, $h = 0.01$, $E = 4$: $|\Delta T| = 2.5 \times 10^{-5}$, $R + T - 1 = 5 \times 10^{-14}$). The reason: the three-term Numerov recursion with real coefficients conserves a discrete Wronskian *exactly*, so the discrete current is conserved whatever the discretisation error. $R + T = 1$ **is a necessary consistency check, not a measure of accuracy.** Accuracy must be measured against the analytical T or by grid refinement. (The corrected scheme couples four points at the two edge nodes and therefore conserves the current only to $O(h^4)$, which is why $R + T - 1$ is of the same size as ΔT in the table above.)

Error analysis and convergence

h	misaligned E=2	aligned $V_0/2$ E=2	corrected E=2	misaligned E=8	aligned $V_0/2$ E=8	corrected E=8
0.1	8.9e-05	1.3e-04	8.4e-06	3.3e-03	6.1e-03	3.1e-04
0.05	2.3e-05	3.9e-05	5.4e-07	8.0e-04	1.6e-03	2.1e-05
0.025	5.7e-06	1.1e-05	3.4e-08	2.0e-04	3.9e-04	1.3e-06
0.0125	1.4e-06	2.8e-06	2.2e-09	5.0e-05	9.9e-05	8.5e-08
0.00625	3.6e-07	7.0e-07	1.4e-10	1.2e-05	2.5e-05	5.3e-09
0.003125	8.9e-08	1.8e-07	7.9e-12	3.1e-06	6.2e-06	3.2e-10
0.0015625	2.2e-08	4.4e-08	5.2e-12	7.8e-07	1.6e-06	5.9e-12
order	**1.99**	**1.90**	**3.98**	**2.01**	**1.98**	**3.96**



Problem 2, convergence of T. A step in the potential reduces Numerov to second order whether or not the edge lies on a node; only the corrected treatment restores the fourth order (slope 4), until the round-off floor.

Conclusions: (i) a discontinuous potential is the most common way to lose the advertised accuracy of Numerov without noticing; (ii) simply placing the edge on a node and using $V_0/2$ does **not** help; (iii) with the one-sided-potential rule and the h^3 jump term the method is again fourth order, and at $h = 0.0125$ it is about 600 times more accurate than the naive scheme.

Part IV — Problem 3: The quantum harmonic oscillator

Physics and analytical solution

$$V(x) = 1/2\, x^2, \qquad \psi'' = -(2E - x^2)\,\psi.$$

Square-integrable solutions exist only for

$$E_n = n + 1/2\,, \qquad \psi_n(x) = \frac{1}{\pi^{1/4}\sqrt{2^n n!}}\, H_n(x)\, e^{-x^2/2}, \qquad n = 0,1,2,\ldots$$

with Hermite polynomials $H_0 = 1$, $H_1 = 2x$, $H_{n+1} = 2xH_n - 2nH_{n-1}$. Useful exact properties for verification:

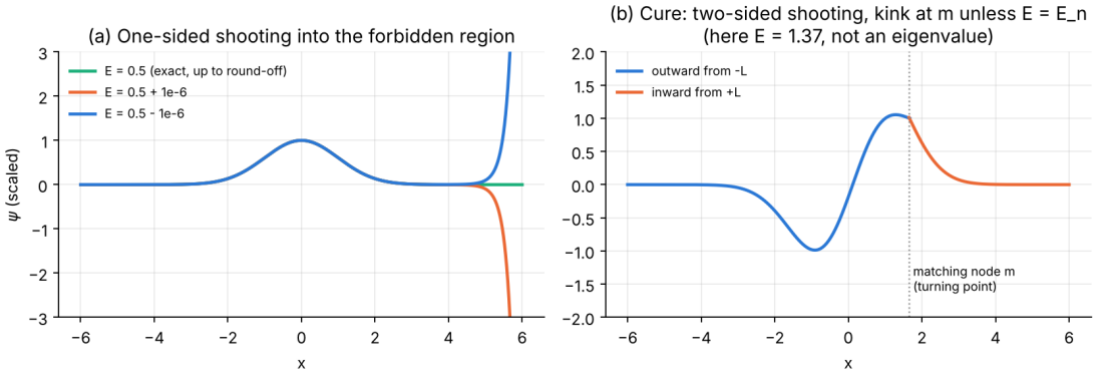
- normalisation and orthogonality $\langle\psi_m|\psi_n\rangle = \delta_{mn}$;
- ψ_n has exactly n nodes (oscillation theorem);
- **virial theorem**: $\langle T\rangle = \langle V\rangle = E_n/2$;
- classical turning points $x_t = \pm\sqrt{2E}$; beyond them $\psi \sim e^{-x^2/2}$ decays.

Numerical scheme

- Grid $x_j = -L + jh$, $h = 2L/N$, on a finite box $[-L, L]$ ($L = 7$ by default); boundary conditions $\psi(\pm L) = 0$, justified because $\psi_n(\pm L)$ is astronomically small.
- **Why not shoot from one side only (fault no. 4)?** Beyond the right turning point the general solution is a mixture of $e^{-x^2/2}$ (physical) and $e^{+x^2/2}$ (unphysical). Any error — round-off or an energy that is wrong by 10^{-6} — excites the growing solution, which dominates exponentially. Integrating from $-L$ all the way to $+L$ is therefore extremely ill-conditioned (Figure below, panel a).
- **Two-sided shooting.** Integrate *outward* from $-L$ ($\psi_0 = 0$, $\psi_1 = \eta$) and *inward* from $+L$ ($\psi_N = 0$, $\psi_{N-1} = \eta$). In both directions we integrate from the forbidden region towards the allowed region, so the physically decaying solution is the *growing* one in the direction of integration — a stable process. The two pieces meet at the node m nearest the right turning point.
- **Matching function.** The two pieces must be proportional to each other. Instead of matching logarithmic derivatives (which has poles where $\psi_m = 0$), we use the **discrete Wronskian** (Casoratian)

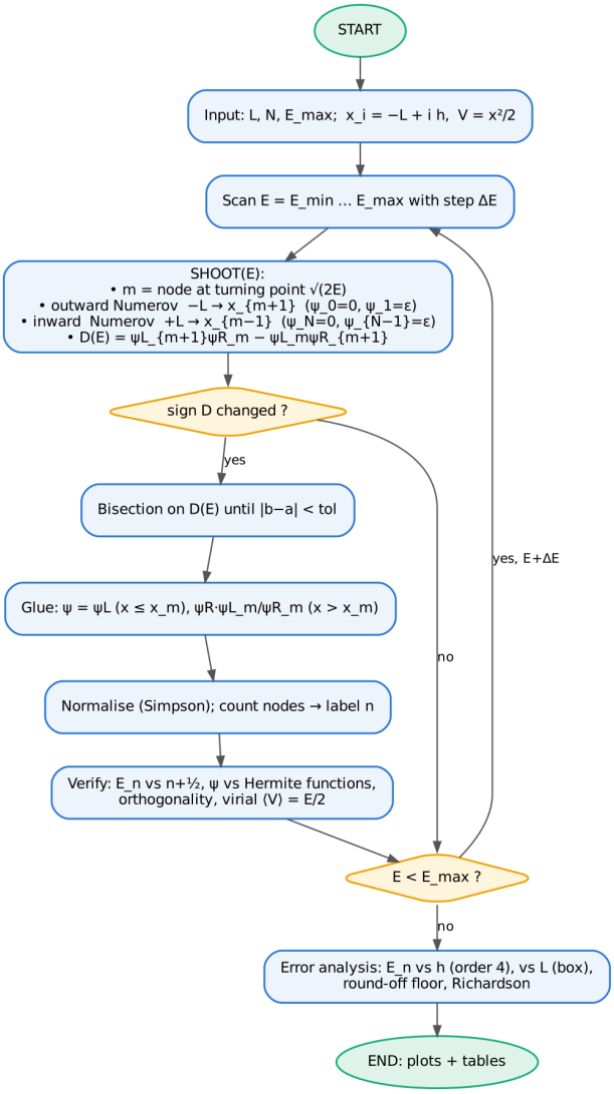
  $$D(E) = \psi^L_{m+1}\psi^R_m - \psi^L_m\psi^R_{m+1}.$$

  Because $u_j = c_j\psi_j$ satisfies $u_{j+1} + u_{j-1} = \left(12/c_j - 10\right) u_j$, the quantity $c_m c_{m+1} D$ is independent of m, and $D = 0$ holds **exactly** when the two discrete solutions coincide — i.e. at the eigenvalue of the discrete problem. $D(E)$ is smooth and has no poles, so a scan + bisection is robust.
- After convergence the pieces are glued at m, normalised with Simpson, the nodes are counted to identify n, and the sign is fixed.



The instability fault and its cure. (a) Shooting from the left only: an energy error of 10^{-6} makes ψ explode beyond the turning point. (b) Two-sided shooting at a trial energy that is not an eigenvalue: the two pieces meet with a kink at the matching node; the kink vanishes when $D(E) = 0$.

Logic chart



Logic chart of the harmonic-oscillator program.

Python 3 code

File: `p3_harmonic_oscillator.py`

```python
#!/usr/bin/env python3
"""
Problem 3 -- QUANTUM HARMONIC OSCILLATOR with the Numerov method
================================================================
Units: hbar = m = omega = 1.   V(x) = x^2 / 2
psi''(x) = -k^2(x) psi(x),  k^2(x) = 2 [E - V(x)] = 2E - x^2
Exact:  E_n = n + 1/2,   psi_n(x) = (pi^-1/4 / sqrt(2^n n!)) H_n(x) exp(-x^2/2)

Eigenvalue method: two-sided shooting ("outward + inward" integration)
  * outward Numerov from x = -L (psi = 0, next point tiny) up to node m+1
  * inward  Numerov from x = +L (psi = 0, next point tiny) down to node m-1
  * m = grid node closest to the right classical turning point x_t = sqrt(2E)
    (integrating INTO a classically forbidden region is unstable: the growing
     exponential swamps the decaying one; we therefore always integrate from
     the forbidden region TOWARDS the allowed one)
  * matching function (discrete Wronskian / Casoratian, has no poles):
        D(E) = psiL[m+1]*psiR[m] - psiL[m]*psiR[m+1]
    D(E) = 0  <=>  the two pieces are proportional  <=>  E is an eigenvalue
  * eigenvalues bracketed by scanning D(E), refined by bisection,
    labelled by counting nodes of the assembled wavefunction.
Verification: normalisation (Simpson), overlap with exact psi_n, orthogonality,
              virial theorem <V> = E/2, Richardson extrapolation.
Error analysis: discretisation error vs h (order 4), truncation error vs box L,
                round-off floor for very small h.
Outputs: tables + p3_wavefunctions.png, p3_convergence.png
"""
import numpy as np
from math import factorial
from numpy.polynomial.hermite import hermval
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4"]


def V(x):
    return 0.5 * x * x


def simpson(y, h):
    n = len(y) - 1
    assert n % 2 == 0, "Simpson needs an even number of intervals"
    return h / 3.0 * (y[0] + y[-1] + 4.0 * y[1:-1:2].sum() + 2.0 * y[2:-1:2].sum())


def psi_exact(n, x):
    coef = np.zeros(n + 1); coef[n] = 1.0
    return np.pi ** -0.25 / np.sqrt(2.0 ** n * factorial(n)) * hermval(x, coef) * np.exp(-x * x / 2)


def make_grid(L, N):
    x = np.linspace(-L, L, N + 1)
    return x, x[1] - x[0]


def shoot(E, x, h):
    """Return (D, psi_assembled, m) for trial energy E."""
    N = len(x) - 1
    k2 = 2.0 * (E - V(x))
    c = 1.0 + h * h * k2 / 12.0
    xt = np.sqrt(max(2.0 * E, 0.0))                    # right turning point
    m = int(np.clip(np.searchsorted(x, xt), 2, N - 2))
    # outward: 0 .. m+1
    pL = np.zeros(m + 2)
    pL[0], pL[1] = 0.0, 1e-10
    for n in range(1, m + 1):
        pL[n + 1] = ((12.0 - 10.0 * c[n]) * pL[n] - c[n - 1] * pL[n - 1]) / c[n + 1]
    # inward: N .. m-1
    pR = np.zeros(N + 1)
    pR[N], pR[N - 1] = 0.0, 1e-10
    for n in range(N - 1, m - 1, -1):
        pR[n - 1] = ((12.0 - 10.0 * c[n]) * pR[n] - c[n + 1] * pR[n + 1]) / c[n - 1]
    # scale to avoid overflow (positive factors do not change the sign of D)
    sL = np.max(np.abs(pL)); sR = np.max(np.abs(pR[m - 1:]))
    pL /= sL; pR /= sR
    D = pL[m + 1] * pR[m] - pL[m] * pR[m + 1]          # discrete Wronskian
    psi = np.empty(N + 1)
    psi[:m + 1] = pL[:m + 1]
    psi[m + 1:] = pR[m + 1:] * (pL[m] / pR[m])          # glue at node m
    return D, psi, m
```

```
def count_nodes(psi):
    p = psi[np.abs(psi) > 1e-6 * np.max(np.abs(psi))]   # ignore numerical tails
    return int(np.sum(p[:-1] * p[1:] < 0))


def eigenstates(nmax=5, L=7.0, N=1400, Emax=None, dE=0.05, tol=1e-14):
    x, h = make_grid(L, N)
    Emax = Emax if Emax else nmax + 1.0
    Egrid = np.arange(0.05, Emax, dE)
    Dg = np.array([shoot(E, x, h)[0] for E in Egrid])
    out = []
    for i in np.where(np.sign(Dg[:-1]) != np.sign(Dg[1:]))[0]:
        a, b, fa = Egrid[i], Egrid[i + 1], Dg[i]
        while b - a > tol * max(1.0, a):                # bisection on D(E)
            mid = 0.5 * (a + b)
            fm = shoot(mid, x, h)[0]
            if np.sign(fm) == np.sign(fa):
                a, fa = mid, fm
            else:
                b = mid
        E = 0.5 * (a + b)
        _, psi, m = shoot(E, x, h)
        psi /= np.sqrt(simpson(psi * psi, h))           # NORMALISATION
        n = count_nodes(psi)
        ex = psi_exact(n, x)
        if np.dot(psi, ex) < 0:
            psi = -psi
        out.append((n, E, psi))
    return x, h, out


def slope(h, e):
    return np.polyfit(np.log(h), np.log(e), 1)[0]


def main():
    print("=" * 78)
    print("Harmonic oscillator  V = x^2/2, box [-7, 7], N = 1400 (h = 0.01)")
    print("=" * 78)
    x, h, st = eigenstates(nmax=5)
    print(f"{'n':>2} {'E_numerov':>18} {'E_exact':>6} {'|dE|':>10} {'norm':>14} "
          f"{'max|psi-ex|':>12} {'<V>':>10} {'E/2':>8}")
    for n, E, psi in st:
        ex = psi_exact(n, x)
        Vexp = simpson(psi * psi * V(x), h)
        print(f"{n:2d} {E:18.12f} {n+0.5:6.1f} {abs(E-n-0.5):10.2e} {simpson(psi*psi,h):14.12f} "
              f"{np.max(abs(psi-ex)):12.2e} {Vexp:10.6f} {E/2:8.4f}")
    P = np.array([s[2] for s in st])
    O = np.array([[simpson(P[i] * P[j], h) for j in range(len(P))] for i in range(len(P))])
    print("max |<psi_i|psi_j> - delta_ij| =", f"{np.max(np.abs(O - np.eye(len(P)))):.2e}")

    # ------------------------------------------- convergence in h (fixed L = 7)
    Ns = [70, 140, 280, 560, 1120, 2240, 4480, 8960]
    hs, errs = [], {0: [], 2: [], 4: []}
    for N in Ns:
        xg, hg, s = eigenstates(nmax=5, L=7.0, N=N)
        hs.append(hg)
        d = {n: E for n, E, _ in s}
        for n in errs:
            errs[n].append(abs(d[n] - (n + 0.5)))
    print("\nEigenvalue error vs step (L = 7):")
    print(f"{'N':>6} {'h':>9}" + "".join(f" {'|dE_'+str(n)+'|':>11} {'ratio':>6}" for n in errs))
    for i, N in enumerate(Ns):
        line = f"{N:6d} {hs[i]:9.5f}"
        for n in errs:
            r = errs[n][i - 1] / errs[n][i] if i else np.nan
            line += f" {errs[n][i]:11.2e} {r:6.1f}"
        print(line)
    for n in errs:
        print(f"  n={n}: observed order p = {slope(hs[:5], errs[n][:5]):.2f}")

    # Richardson extrapolation for E_4
    _, _, s1 = eigenstates(L=7.0, N=140); _, _, s2 = eigenstates(L=7.0, N=280)
    E1 = dict((n, E) for n, E, _ in s1)[4]; E2 = dict((n, E) for n, E, _ in s2)[4]
    ER = (16 * E2 - E1) / 15
    print(f"\nRichardson n=4: E(h=0.1)={E1:.10f}  E(h=0.05)={E2:.10f}  E_R={ER:.10f}  err={abs(ER-4.5):.1e}")

    # ------------------------------------------- truncation error vs L (h = 0.005)
    Ls = [3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 7.0]
    errL = {0: [], 2: [], 4: []}
    for L in Ls:
```

```
        N = int(round(2 * L / 0.005)); N += N % 2
        _, _, s = eigenstates(nmax=5, L=L, N=N, Emax=6.0)
        d = {n: E for n, E, _ in s}
        for n in errL:
            errL[n].append(abs(d.get(n, np.nan) - (n + 0.5)))
    print("\nBox-truncation error (h = 0.005):")
    print(f"{'L':>5}" + "".join(f" {'|dE_'+str(n)+'|':>11}" for n in errL))
    for i, L in enumerate(Ls):
        print(f"{L:5.1f}" + "".join(f" {errL[n][i]:11.2e}" for n in errL))

    # ------------------------------------------- figures
    fig, ax = plt.subplots(1, 2, figsize=(10, 4.2))
    xx = np.linspace(-5, 5, 400)
    ax[0].plot(xx, V(xx), color="#888888", lw=1)
    for i, (n, E, psi) in enumerate(st[:5]):
        sel = (x > -5) & (x < 5)
        ax[0].axhline(E, color="#dddddd", lw=.8)
        ax[0].plot(x[sel], E + 0.6 * psi[sel], color=C[i], lw=2, label=f"n={n}, E={E:.6f}")
        ax[0].plot(x[sel][::40], E + 0.6 * psi_exact(n, x[sel][::40]), "o", ms=3, color="#333333")
    ax[0].set(xlim=(-5, 5), ylim=(0, 7.2), xlabel="x", ylabel=r"$E_n + 0.6\,\psi_n(x)$",
              title="(a) Normalised eigenfunctions (dots: exact)")
    ax[0].legend(frameon=False, fontsize=7, loc="upper center", ncol=2)
    for i, (n, E, psi) in enumerate(st[:5]):
        ax[1].semilogy(x, np.abs(psi - psi_exact(n, x)) + 1e-18, color=C[i], lw=1.2, label=f"n={n}")
    ax[1].set(xlim=(-7, 7), ylim=(1e-14, 1e-5), xlabel="x", ylabel=r"$|\psi_{num}-\psi_{exact}|$",
              title="(b) Pointwise error, h = 0.01")
    ax[1].legend(frameon=False, fontsize=7, ncol=5, loc="upper center")
    for a in ax:
        a.spines[["top", "right"]].set_visible(False); a.grid(alpha=.25)
    fig.tight_layout(); fig.savefig("p3_wavefunctions.png", dpi=170)

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.9))
    hs_a = np.array(hs)
    for i, n in enumerate(errs):
        ax[0].loglog(hs_a, errs[n], "o-", color=C[i], lw=2, ms=5, label=f"$E_{n}$")
    ax[0].loglog(hs_a, errs[4][0] * (hs_a / hs_a[0]) ** 4, ":", color="#888888", label=r"$\propto h^4$")
    ax[0].set(xlabel="step h", ylabel=r"$|E_n^{num}-(n+1/2)|$",
              title="(a) Discretisation error (L = 7)")
    ax[0].legend(frameon=False, fontsize=8)
    for i, n in enumerate(errL):
        ax[1].semilogy(Ls, errL[n], "o-", color=C[i], lw=2, ms=5, label=f"$E_{n}$")
    ax[1].set(xlabel="half-width of box L", ylabel=r"$|E_n^{num}-(n+1/2)|$",
              title="(b) Truncation error of the box (h = 0.005)")
    ax[1].legend(frameon=False, fontsize=8)
    for a in ax:
        a.spines[["top", "right"]].set_visible(False); a.grid(alpha=.25, which="both")
    fig.tight_layout(); fig.savefig("p3_convergence.png", dpi=170)
    print("\nfigures written: p3_wavefunctions.png, p3_convergence.png")


if __name__ == "__main__":
    main()
```

Fortran 90 code

File: `p3_harmonic_oscillator.f90`

```
!=======================================================================
! Problem 3 -- QUANTUM HARMONIC OSCILLATOR with the Numerov method
! Units hbar = m = omega = 1.  V = x^2/2,  exact E_n = n + 1/2
! psi'' = -k^2(x) psi ,  k^2 = 2E - x^2
!
! Two-sided shooting: outward from -L, inward from +L, matched at the
! node m nearest to the right classical turning point sqrt(2E).
! Matching function = discrete Wronskian (no poles):
!     D(E) = psiL(m+1) psiR(m) - psiL(m) psiR(m+1)
! Bracketing by an energy scan, refinement by bisection, labelling by
! node counting.  Verification: Simpson normalisation, comparison with
! Hermite functions, orthogonality, virial theorem <V> = E/2.
! Error analysis: E_n error vs h (order 4), vs box half-width L.
! Output : screen tables + p3_states.dat, p3_conv_h.dat, p3_conv_L.dat
! Compile: gfortran -O2 -o p3 p3_harmonic_oscillator.f90 && ./p3
!=======================================================================
module oscillator
  implicit none
  integer,  parameter :: dp = kind(1.0d0)
  real(dp), parameter :: pi = 3.141592653589793238_dp
  integer,  parameter :: maxst = 10
contains
  pure elemental function V(x)
```

```
  real(dp), intent(in) :: x
  real(dp) :: V
  V = 0.5_dp*x*x
end function V

function simpson(y, h, n) result(s)
  integer,  intent(in) :: n
  real(dp), intent(in) :: y(0:n), h
  real(dp) :: s
  s = h/3.0_dp*( y(0) + y(n) + 4.0_dp*sum(y(1:n-1:2)) + 2.0_dp*sum(y(2:n-2:2)) )
end function simpson

! exact normalised eigenfunction via the Hermite recurrence
function psi_exact(n, x) result(p)
  integer,  intent(in) :: n
  real(dp), intent(in) :: x
  real(dp) :: p, H0, H1, H2, fact
  integer  :: j
  H0 = 1.0_dp; H1 = 2.0_dp*x; H2 = H0
  if (n == 0) then
     H2 = H0
  else if (n == 1) then
     H2 = H1
  else
     do j = 2, n
        H2 = 2.0_dp*x*H1 - 2.0_dp*(j-1)*H0
        H0 = H1; H1 = H2
     end do
  end if
  fact = 1.0_dp
  do j = 2, n
     fact = fact*j
  end do
  p = pi**(-0.25_dp)/sqrt(2.0_dp**n*fact)*H2*exp(-x*x/2.0_dp)
end function psi_exact

!-----------------------------------------------------------------
! shooting for trial energy E on grid x(0:N); returns D and the
! assembled (unnormalised) wavefunction
!-----------------------------------------------------------------
subroutine shoot(E, x, h, N, D, psi)
  integer,  intent(in)  :: N
  real(dp), intent(in)  :: E, x(0:N), h
  real(dp), intent(out) :: D, psi(0:N)
  real(dp) :: c(0:N), pL(0:N), pR(0:N), sL, sR
  integer  :: m, j
  c = 1.0_dp + h*h*2.0_dp*(E - V(x))/12.0_dp
  m = 0
  do j = 0, N                                  ! first node >= turning point
     if (x(j) >= sqrt(max(2.0_dp*E, 0.0_dp))) then
        m = j; exit
     end if
  end do
  m = max(2, min(m, N-2))
  pL = 0.0_dp; pR = 0.0_dp
  pL(0) = 0.0_dp; pL(1) = 1.0d-10              ! outward
  do j = 1, m
     pL(j+1) = ((12.0_dp - 10.0_dp*c(j))*pL(j) - c(j-1)*pL(j-1))/c(j+1)
  end do
  pR(N) = 0.0_dp; pR(N-1) = 1.0d-10            ! inward
  do j = N-1, m, -1
     pR(j-1) = ((12.0_dp - 10.0_dp*c(j))*pR(j) - c(j+1)*pR(j+1))/c(j-1)
  end do
  sL = maxval(abs(pL(0:m+1))); sR = maxval(abs(pR(m-1:N)))
  pL = pL/sL; pR = pR/sR
  D = pL(m+1)*pR(m) - pL(m)*pR(m+1)            ! discrete Wronskian
  psi(0:m)   = pL(0:m)
  psi(m+1:N) = pR(m+1:N)*(pL(m)/pR(m))
end subroutine shoot

integer function nodes(psi, N)
  integer,  intent(in) :: N
  real(dp), intent(in) :: psi(0:N)
  real(dp) :: thr, last
  integer  :: j
  thr = 1.0d-6*maxval(abs(psi)); nodes = 0; last = 0.0_dp
  do j = 0, N
     if (abs(psi(j)) > thr) then
        if (last*psi(j) < 0.0_dp) nodes = nodes + 1
        last = psi(j)
     end if
  end do
```

```
  end function nodes

  !-------------------------------------------------------------
  ! all eigenstates with E < Emax on [-L,L] with N intervals
  !-------------------------------------------------------------
  subroutine eigenstates(L, N, Emax, nfound, nlab, Eval, states)
    real(dp), intent(in)  :: L, Emax
    integer,  intent(in)  :: N
    integer,  intent(out) :: nfound, nlab(maxst)
    real(dp), intent(out) :: Eval(maxst), states(0:N, maxst)
    real(dp) :: x(0:N), psi(0:N), h, E, Eold, D, Dold, a, b, fa, mid, fm, s, dE
    integer  :: j, nn
    h = 2.0_dp*L/N; dE = 0.05_dp
    x = [(-L + j*h, j=0,N)]
    nfound = 0
    Eold = 0.05_dp; call shoot(Eold, x, h, N, Dold, psi)
    E = Eold
    do while (E + dE < Emax)
       E = E + dE
       call shoot(E, x, h, N, D, psi)
       if (sign(1.0_dp,D) /= sign(1.0_dp,Dold)) then
          a = E - dE; b = E; fa = Dold
          do while (b - a > 1.0d-14*max(1.0_dp, a))   ! bisection
             mid = 0.5_dp*(a + b)
             call shoot(mid, x, h, N, fm, psi)
             if (sign(1.0_dp,fm) == sign(1.0_dp,fa)) then
                a = mid; fa = fm
             else
                b = mid
             end if
          end do
          call shoot(0.5_dp*(a+b), x, h, N, fm, psi)
          psi = psi/sqrt(simpson(psi*psi, h, N))      ! normalisation
          nn = nodes(psi, N)
          s = 0.0_dp
          do j = 0, N
             s = s + psi(j)*psi_exact(nn, x(j))
          end do
          if (s < 0.0_dp) psi = -psi
          nfound = nfound + 1
          nlab(nfound) = nn; Eval(nfound) = 0.5_dp*(a+b); states(:,nfound) = psi
       end if
       Dold = D
    end do
  end subroutine eigenstates

  function order(h, e, m) result(p)
    integer,  intent(in) :: m
    real(dp), intent(in) :: h(m), e(m)
    real(dp) :: p, lx(m), ly(m)
    lx = log(h); ly = log(e)
    p = (m*sum(lx*ly) - sum(lx)*sum(ly)) / (m*sum(lx*lx) - sum(lx)**2)
  end function order

  real(dp) function Efor(nwant, nfound, nlab, Eval)
    integer,  intent(in) :: nwant, nfound, nlab(maxst)
    real(dp), intent(in) :: Eval(maxst)
    integer :: j
    Efor = huge(1.0_dp)
    do j = 1, nfound
       if (nlab(j) == nwant) Efor = Eval(j)
    end do
  end function Efor
end module oscillator

program harmonic_oscillator
  use oscillator
  implicit none
  integer, parameter :: NL = 1400, nh = 8, nLs = 8
  integer  :: Ns(nh) = [70, 140, 280, 560, 1120, 2240, 4480, 8960]
  real(dp) :: Ls(nLs) = [3.0_dp, 3.5_dp, 4.0_dp, 4.5_dp, 5.0_dp, 5.5_dp, 6.0_dp, 7.0_dp]
  integer  :: wanted(3) = [0, 2, 4]
  real(dp), allocatable :: st(:,:), x(:)
  real(dp) :: Eval(maxst), h, Vexp, ex, maxerr, hs(nh), errs(3,nh), errL(3,nLs), omax, ov
  real(dp) :: L, E1, E2, ER
  integer  :: nlab(maxst), nf, i, j, q, N

  L = 7.0_dp; h = 2.0_dp*L/NL
  allocate(st(0:NL, maxst), x(0:NL))
  x = [(-L + j*h, j=0,NL)]
  call eigenstates(L, NL, 6.0_dp, nf, nlab, Eval, st)
  print '(a)', repeat('=',78)
```

```
  print '(a)', 'Harmonic oscillator  V = x^2/2, box [-7, 7], N = 1400 (h = 0.01)'
  print '(a)', repeat('=',78)
  print '(a2,a18,a8,a11,a16,a13,a11,a9)', 'n', 'E_numerov', 'E_exact', '|dE|', 'norm', &
       'max|psi-ex|', '<V>', 'E/2'
  open(10, file='p3_states.dat', status='replace')
  do i = 1, nf
     maxerr = 0.0_dp
     do j = 0, NL
        ex = psi_exact(nlab(i), x(j)); maxerr = max(maxerr, abs(st(j,i) - ex))
     end do
     Vexp = simpson(st(:,i)**2*V(x), h, NL)
     print '(i2,f18.12,f8.1,es11.2,f16.12,es13.2,f11.6,f9.4)', nlab(i), Eval(i), nlab(i)+0.5_dp, &
          abs(Eval(i) - nlab(i) - 0.5_dp), simpson(st(:,i)**2, h, NL), maxerr, Vexp, Eval(i)/2
  end do
  do j = 0, NL
     write(10,'(7es16.7)') x(j), st(j,1:nf)
  end do
  close(10)
  omax = 0.0_dp
  do i = 1, nf
     do q = 1, nf
        ov = simpson(st(:,i)*st(:,q), h, NL)
        if (i == q) ov = ov - 1.0_dp
        omax = max(omax, abs(ov))
     end do
  end do
  print '(a,es9.2)', 'max |<psi_i|psi_j> - delta_ij| = ', omax
  deallocate(st, x)

  ! ---------------- convergence in h
  open(11, file='p3_conv_h.dat', status='replace')
  print '(/a)', 'Eigenvalue error vs step (L = 7):'
  print '(a6,a9,3a12)', 'N', 'h', '|dE_0|', '|dE_2|', '|dE_4|'
  do i = 1, nh
     N = Ns(i); hs(i) = 2.0_dp*L/N
     allocate(st(0:N, maxst))
     call eigenstates(L, N, 6.0_dp, nf, nlab, Eval, st)
     do q = 1, 3
        errs(q,i) = abs(Efor(wanted(q), nf, nlab, Eval) - wanted(q) - 0.5_dp)
     end do
     print '(i6,f9.5,3es12.2)', N, hs(i), errs(:,i)
     write(11,'(4es16.7)') hs(i), errs(:,i)
     deallocate(st)
  end do
  close(11)
  print '(a,3f6.2)', 'observed order p (n=0,2,4): ', (order(hs(1:5), errs(q,1:5), 5), q=1,3)

  allocate(st(0:280, maxst))
  call eigenstates(L, 140, 6.0_dp, nf, nlab, Eval, st(0:140,:))
  E1 = Efor(4, nf, nlab, Eval)
  call eigenstates(L, 280, 6.0_dp, nf, nlab, Eval, st)
  E2 = Efor(4, nf, nlab, Eval)
  ER = (16.0_dp*E2 - E1)/15.0_dp
  print '(/a,f13.10,a,f13.10,a,f13.10,a,es8.1)', 'Richardson n=4: E(0.1)=', E1, '  E(0.05)=', E2, &
       '  E_R=', ER, '  err=', abs(ER - 4.5_dp)
  deallocate(st)

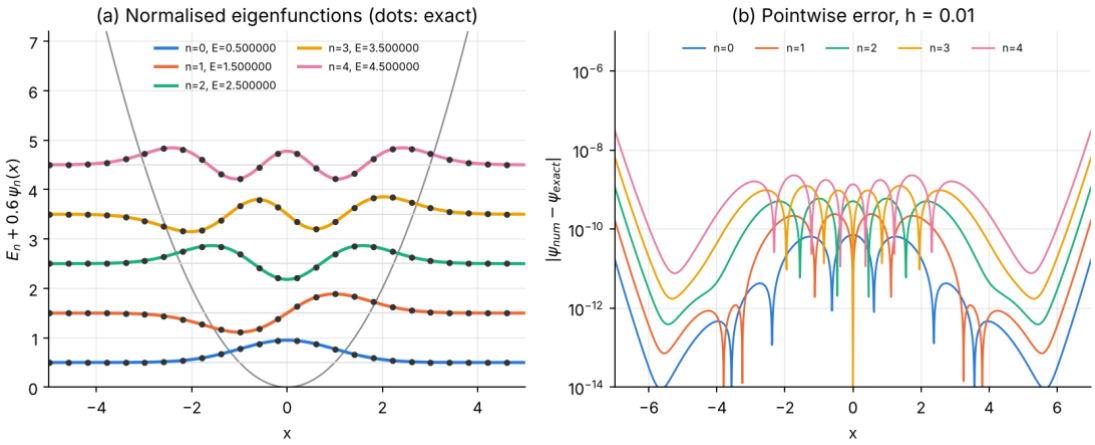
  ! ---------------- truncation error vs L
  open(12, file='p3_conv_L.dat', status='replace')
  print '(/a)', 'Box-truncation error (h = 0.005):'
  print '(a5,3a12)', 'L', '|dE_0|', '|dE_2|', '|dE_4|'
  do i = 1, nLs
     N = nint(2.0_dp*Ls(i)/0.005_dp); N = N + mod(N,2)
     allocate(st(0:N, maxst))
     call eigenstates(Ls(i), N, 6.0_dp, nf, nlab, Eval, st)
     do q = 1, 3
        errL(q,i) = abs(Efor(wanted(q), nf, nlab, Eval) - wanted(q) - 0.5_dp)
     end do
     print '(f5.1,3es12.2)', Ls(i), errL(:,i)
     write(12,'(4es16.7)') Ls(i), errL(:,i)
     deallocate(st)
  end do
  close(12)
end program harmonic_oscillator
```

Results, normalisation and verification

Box $[-7,7]$, $N = 1400$ ($h = 0.01$):

n	E_n Numerov	exact	$\lvert\Delta E\rvert$	$\int\psi^2 dx$	$\max\lvert\psi - \psi_{ex}\rvert$	$\langle V\rangle$	$E_n/2$
0	0.499999999961	0.5	3.9e-11	1.000000000000	7.1e-11	0.250000	0.2500
1	1.499999999727	1.5	2.7e-10	1.000000000000	2.4e-10	0.750000	0.7500
2	2.499999999023	2.5	9.8e-10	1.000000000000	1.2e-09	1.250000	1.2500
3	3.499999997539	3.5	2.5e-09	1.000000000000	6.6e-09	1.750000	1.7500
4	4.499999994962	4.5	5.0e-09	1.000000000000	3.2e-08	2.250000	2.2500
5	5.499999990976	5.5	9.0e-09	1.000000000000	1.3e-07	2.750000	2.7500

All checks pass: the energies agree with $n + 1/2$ to 8–10 digits, every state is normalised, the largest deviation of the overlap matrix from the identity is 3×10^{-13}, each state has the right number of nodes, and the virial theorem $\langle V\rangle = E_n/2$ is satisfied to six printed digits.



Problem 3. (a) The normalised numerical eigenfunctions (lines) and the exact Hermite functions (dots), drawn at the height of their energy inside the potential. (b) Pointwise error; it is largest for the highest state and in the classically forbidden tails.

Error analysis and convergence

Three error sources are separated.

(a) Discretisation error (vary h at fixed $L = 7$):

N	h	$\|\Delta E_0\|$	ratio	$\|\Delta E_2\|$	ratio	$\|\Delta E_4\|$	ratio
70	0.2	6.28e-06	—	1.58e-04	—	8.18e-04	—
140	0.1	3.91e-07	16.1	9.79e-06	16.1	5.06e-05	16.2
280	0.05	2.44e-08	16.0	6.11e-07	16.0	3.15e-06	16.0
560	0.025	1.53e-09	16.0	3.82e-08	16.0	1.97e-07	16.0
1120	0.0125	9.57e-11	15.9	2.38e-09	16.0	1.23e-08	16.0
2240	0.00625	6.20e-12	15.4	1.50e-10	15.9	7.69e-10	16.0
4480	0.003125	3.42e-12	1.8	1.16e-11	12.9	4.83e-11	15.9
8960	0.0015625	6.02e-12	0.6	6.95e-12	1.7	5.45e-12	8.9

The ratio 16 (order $p = 4.00$ for all three states) holds over five halvings; then the round-off floor ($\sim 5 \times 10^{-12}$) is reached.

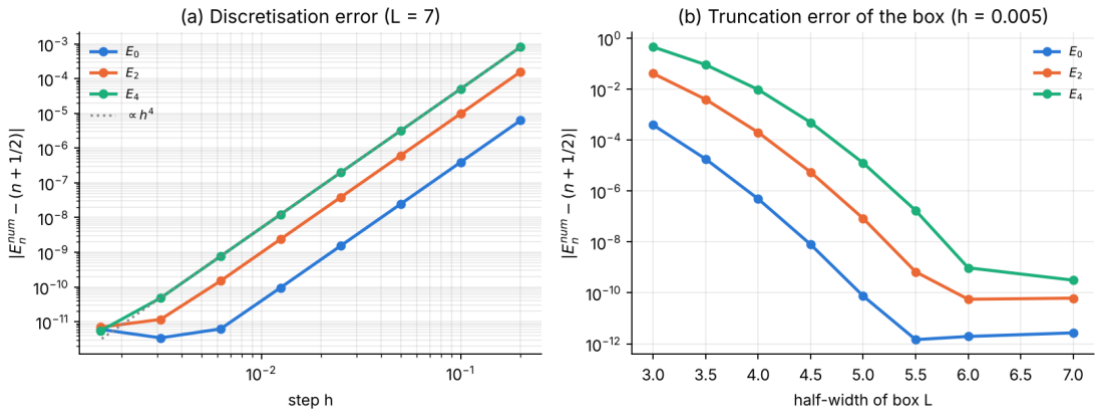
(b) Truncation (box-size) error (vary L at fixed $h = 0.005$): replacing $\psi(\pm\infty) = 0$ by $\psi(\pm L) = 0$ raises the energy by an amount proportional to $|\psi_n(L)|^2 \sim e^{-L^2}$.

L	$\|\Delta E_0\|$	$\|\Delta E_2\|$	$\|\Delta E_4\|$
3.0	3.9e-04	4.1e-02	4.5e-01
4.0	4.9e-07	2.0e-04	9.6e-03
5.0	7.5e-11	8.4e-08	1.3e-05
6.0	1.9e-12	5.6e-11	9.7e-10
7.0	2.7e-12	6.2e-11	3.2e-10

Higher states extend further ($x_t = \sqrt{2n+1}$) and need a larger box. Rule: choose L a few units beyond the outermost turning point of the highest state required. Beyond $L \approx 6$ the remaining error is the discretisation error of panel (a).

(c) Round-off error — visible as the floor in panel (a).

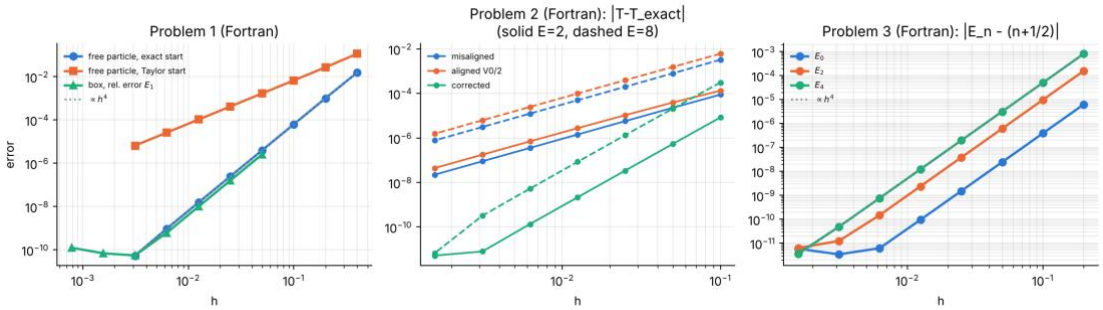
Richardson extrapolation for E_4: $E(h = 0.1) = 4.4999494339$, $E(h = 0.05) = 4.4999968479$, giving $E_R = 4.5000000088$ (error 9×10^{-9} instead of 3×10^{-6}).



Problem 3, convergence. (a) Error of E_0, E_2, E_4 versus step: slope 4 down to the round-off floor. (b) Error versus box half-width: the Gaussian decay of the boundary error.

Part V — Cross-check of the Fortran and Python implementations

All three Fortran programs reproduce the Python tables digit for digit (the only differences are in the last digit of quantities at the round-off level, e.g. 1.2×10^{-12} vs 1.5×10^{-12}). The Fortran programs write their data to `.dat` files; `plot_fortran_results.py` draws the following figure from them.



Convergence plots produced from the Fortran output files. Left: Problem 1; centre: Problem 2 (three treatments of the discontinuity); right: Problem 3.

Part VI — Summary

	Free particle	Square barrier	Harmonic oscillator
Spectrum	continuous (discrete in a box)	continuous	discrete
Numerov use	initial value / shooting to $\psi(L) = 0$	backward complex integration	two-sided shooting + matching
Normalisation	Simpson (box); none possible on $\mathbb{R}$	incident amplitude, flux $R + T = 1$	Simpson
Main fault met	starting values	discontinuous V	instability in forbidden region
Observed order	4 (2 with Taylor start)	2 naive, 4 corrected	4
Best accuracy reached	$\sim 10^{-11}$	$\sim 10^{-12}$	$\sim 10^{-12}$

Key messages:

1. Numerov is a cheap fourth-order integrator for $\psi'' = -k^2\psi$; the observed ratio 16 on halving h is the signature of correct implementation.
2. The order is only realised if the starting values are accurate enough, the potential is smooth (or its discontinuities are treated explicitly) and integration never proceeds into a classically forbidden region.
3. Normalisation, orthogonality, the virial theorem and current conservation are necessary checks — but only comparison with exact results or grid refinement measures the *error*.
4. Grid refinement cannot go below the round-off floor; Richardson extrapolation is the cheap way to gain extra digits.

Part VII — Exercises

1. **Order from two grids.** Using the Problem 1 Part A table, compute $p = \log_2[\varepsilon(h)/\varepsilon(h/2)]$ for every pair of rows and explain the last value (17.6).
2. **A better start.** In Part A use the third-order Taylor start $\psi_1 = kh - (kh)^3/6$. Predict, then measure, the global order.
3. **Points per wavelength.** For the box, find the smallest N that gives E_{10} with relative error $< 10^{-6}$. Express the result as points per wavelength and compare with the rule of thumb of Part I.
4. **Dispersion relation.** Replace e^{ikx} by $e^{ik_h x}$ (with the discrete k_h of Part I) in the starting values and in the extraction of A, B in Problem 2. Does the error decrease? Why?
5. **Thick barrier.** Compute T for $V_0 = 5$, $a = 6$, $E = 1$. Compare with the WKB estimate $T \approx e^{-2\kappa a}$. What limits the accuracy, and what happens to $R + T$?
6. **Smooth barrier.** Replace the square barrier by $V(x) = V_0/\cosh^2(x/w)$, whose exact T is known. Show that no correction is needed to obtain fourth order.
7. **One-sided shooting.** Modify Problem 3 to integrate from $-L$ to $+L$ and require $\psi_N = 0$. For which L does the bisection still converge to E_0? Explain with fault no. 4.
8. **Anharmonic oscillator.** Use the Problem 3 code for $V = x^2/2 + \lambda x^4$ with $\lambda = 0.1$. Check against first-order perturbation theory $\Delta E_n = 3\lambda/4\,(2n^2 + 2n + 1)$ for small λ.
9. **Finite square well.** Turn Problem 2 into a bound-state problem with $V = -V_0$ for $|x| < a/2$ using the shooting/matching method of Problem 3 and the discontinuity correction of Problem 2. Verify the even-state condition $q\tan(qa/2) = \kappa$ with $q = \sqrt{2(V_0 - |E|)}$, $\kappa = \sqrt{2|E|}$.
10. **Round-off.** Repeat the E_0 study of Problem 3 in single precision (Fortran `real(4)`) and locate the new optimum h.

References

- B. V. Numerov, “A method of extrapolation of perturbations”, *Monthly Notices of the Royal Astronomical Society* **84**, 592 (1924).
- B. V. Numerov, “Note on the numerical integration of $d^2x/dt^2 = f(x,t)$”, *Astronomische Nachrichten* **230**, 359 (1927).
- J. M. Blatt, “Practical points concerning the solution of the Schrödinger equation”, *J. Comput. Phys.* **1**, 382 (1967).
- S. E. Koonin and D. C. Meredith, *Computational Physics* (Addison-Wesley, 1990), ch. 3.
- J. M. Thijssen, *Computational Physics*, 2nd ed. (Cambridge University Press, 2007), app. A.
- P. Giannozzi, *Numerical Methods in Quantum Mechanics*, lecture notes, University of Udine.
- D. J. Griffiths and D. F. Schroeter, *Introduction to Quantum Mechanics*, 3rd ed. (Cambridge, 2018), ch. 2.